Table of Contents

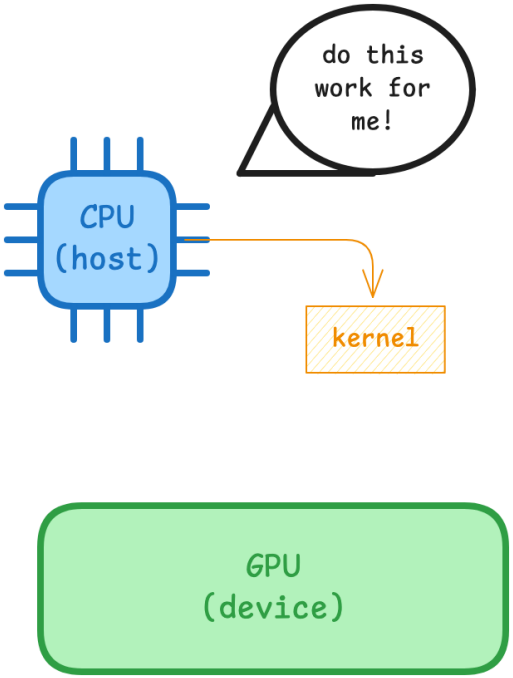
1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

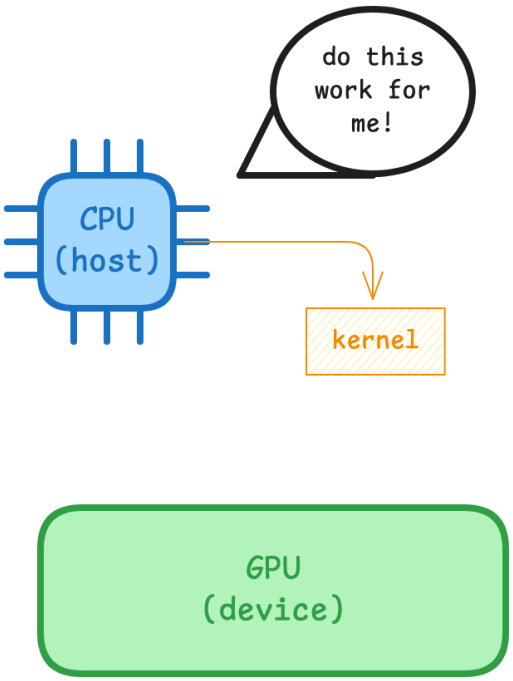
CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.



CUDA Basics (Host vs. Device)

- The CPU is known as the **host**, and manages top level logic.
- The GPU is known as the **device**, and does the heavy lifting.
- The host launches **kernels** to the device.




```
int main() {
  const int M = 128, N = 128, K = 128;

  float *dA, *dB, *dC;
  cudaMalloc(&dA, M * K * sizeof(float);
  cudaMalloc(&dB, K * N * sizeof(float));
  cudaMalloc(&dC, M * N * sizeof(float));

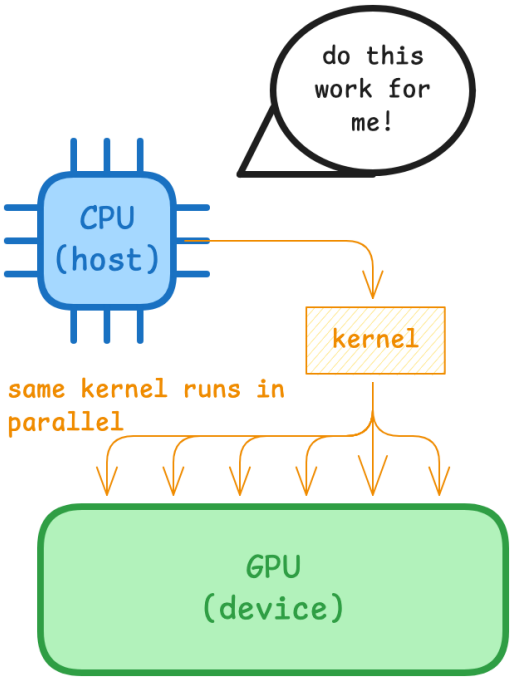
  // load values into these device pointers...

  run_sgemm_naive(M, N, K, 1, dA, dB, 0, dC);

  cudaFree(dA); cudaFree(dB); cudaFree(dC);
}
```

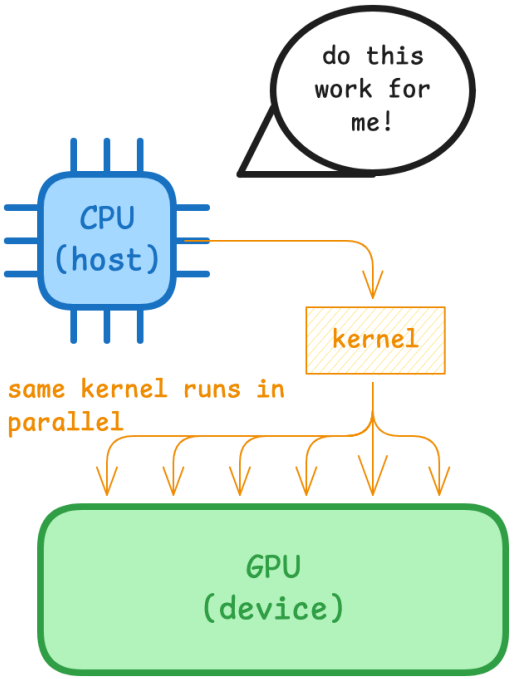
CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.



CUDA Basics (Kernel Execution)

- A kernel runs in parallel on many **threads** on the device.
- The number of threads the device allocates is configured by the host, when it **launches** the kernel.

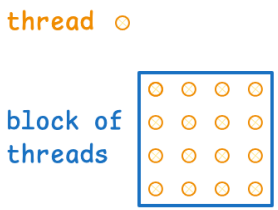



```cpp
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

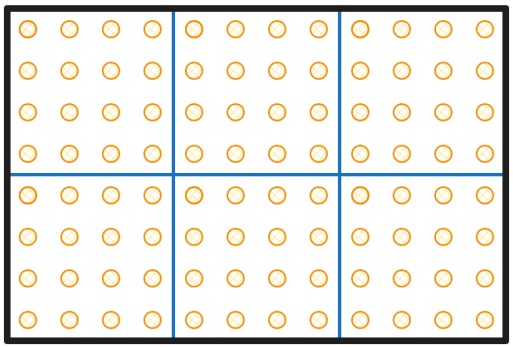
```cpp
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

CUDA Basics (Threads, Blocks, Grids)

- Threads are grouped into **blocks**.
- The collection of blocks requested by the host is organized in a **grid**.



Grid of blocks



```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  // implementation hidden for now ...
}
```

```
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution. We will see later how to use this to optimize performance.

Why Threads, Blocks, Grids?

This seems complicated, why can't we just ask for N threads?

Relation to hardware

This hierarchy is closely related to how the device schedules the execution.
We will see later how to use this to optimize performance.

Thread, Block, Grid hierarchy

Threads are grouped into blocks of constant size/shape.
Blocks are launched as a grid.
We choose the dimensions of the grid to control how many blocks are launched, and thus how many threads are launched.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

Example

During the forward pass when training a model, we accumulate the gradients from the entire batch.

What is GEMM?

Definition: Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.

```
sgemm(
  int M, int N, int K,
  float alpha, float *A, float *B,
  float beta, float *C
)
```

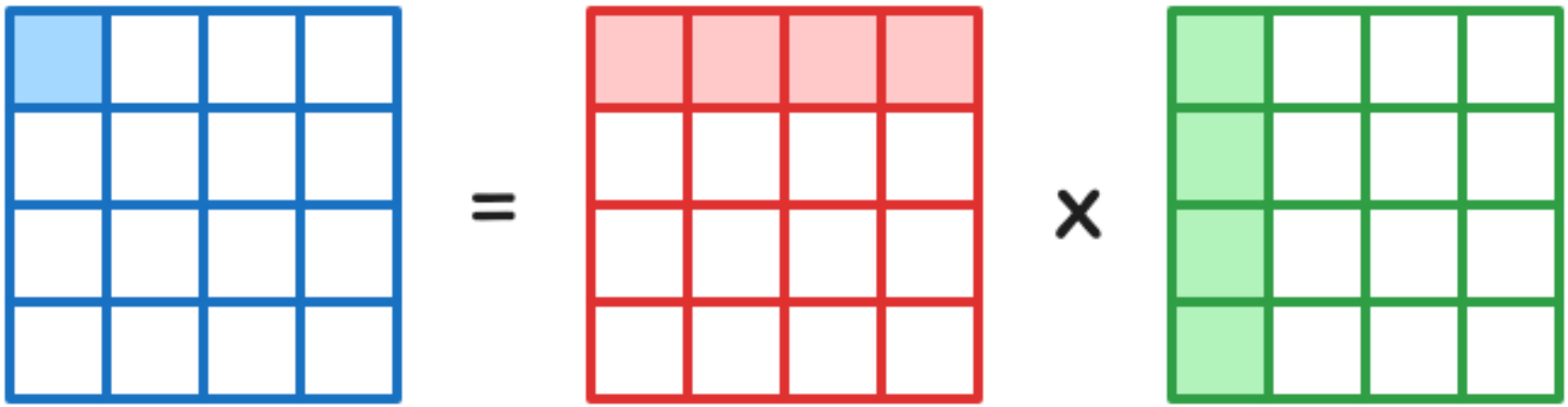
$$\underbrace{A}_{M \times K} \quad \underbrace{B}_{K \times N} \quad \underbrace{C}_{M \times N}$$

- M: rows of A and C
- N: columns of B and C
- K: shared reduction dimension

Let's design a SGEMM kernel!

Recall

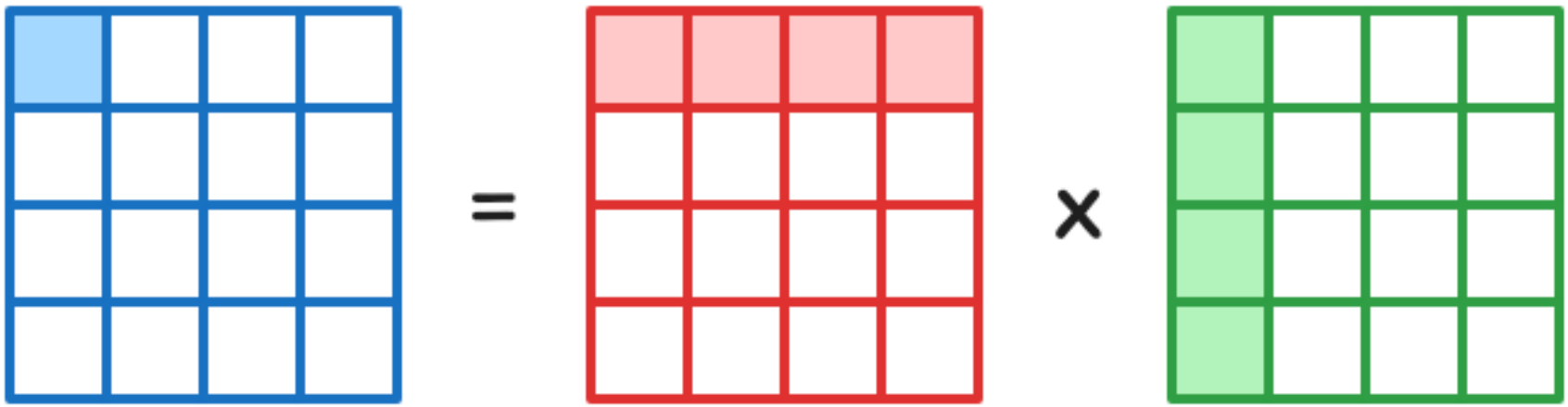
Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.



Let's design a SGEMM kernel!

Recall

Recall that matrix multiplication is naturally parallel, and the output elements of C can be computed independently in parallel.



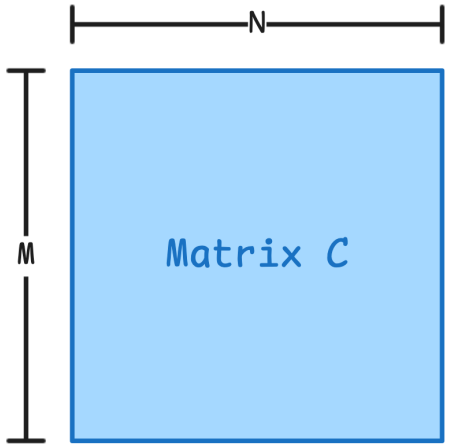
We can naturally map one thread to each output element!

Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

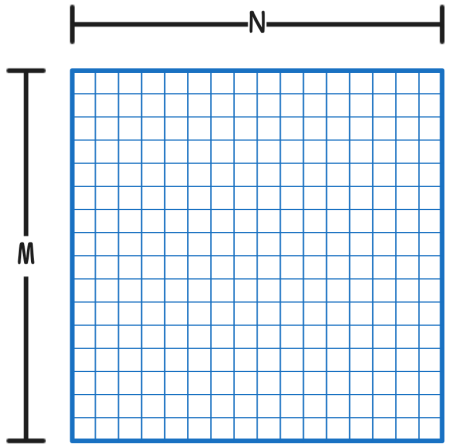


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

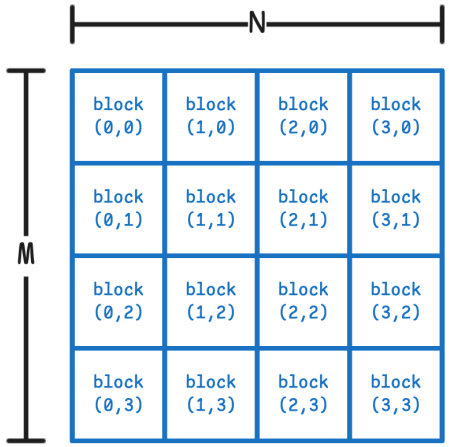


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

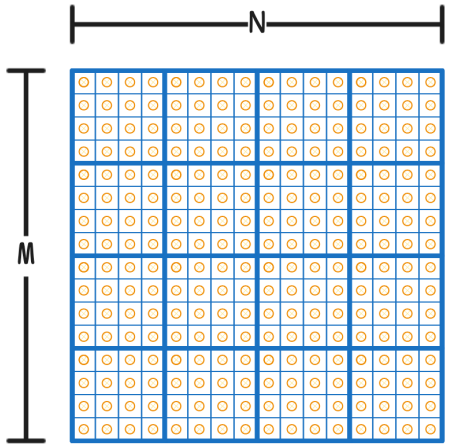


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```

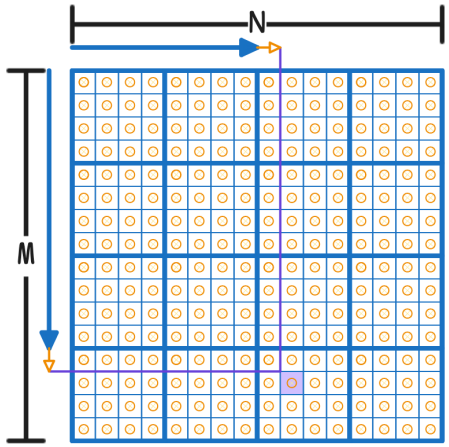


Naive SGEMM Kernel

```
__global__ void sgemm_naive(const int M, const int N, const int K,
                            const float alpha, const float *A, const float *B,
                            const float beta, float *C) {
  const int y = blockIdx.y * blockDim.y + threadIdx.y;
  const int x = blockIdx.x * blockDim.x + threadIdx.x;

  if (y < M && x < N) {
    float sum = 0;
    for (int i = 0; i < K; i++) {
      sum += A[y * K + i] * B[i * N + x];
    }

    C[y * N + x] = alpha * sum + beta * C[y * N + x];
  }
}
template <int BM, int BN>
void launch_sgemm_naive(const int M, const int N, const int K,
                        const float alpha, const float *A, const float *B,
                        const float beta, float *C) {
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}
```



Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.

Play naive-kernel visualization

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
int M = 5000, N = 4000, K = 3000;
float alpha = 1, beta = 0;

float *dA, *dB, *dC; // Allocate buffers on the DEVICE.
cudaMalloc(&dA, M * K * sizeof(float);
cudaMalloc(&dB, K * N * sizeof(float));
cudaMalloc(&dC, M * N * sizeof(float));
// BM, BN are compile-time constants!
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
```

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(BN, BM);
  dim3 gridDim((N + BN - 1) / BN, (M + BM - 1) / BM);
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Naive kernel launch

Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(32, 32);
  dim3 gridDim((4000 + 32 - 1) / 32, (5000 + 32 - 1) / 32); // Ceil division
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Naive kernel launch

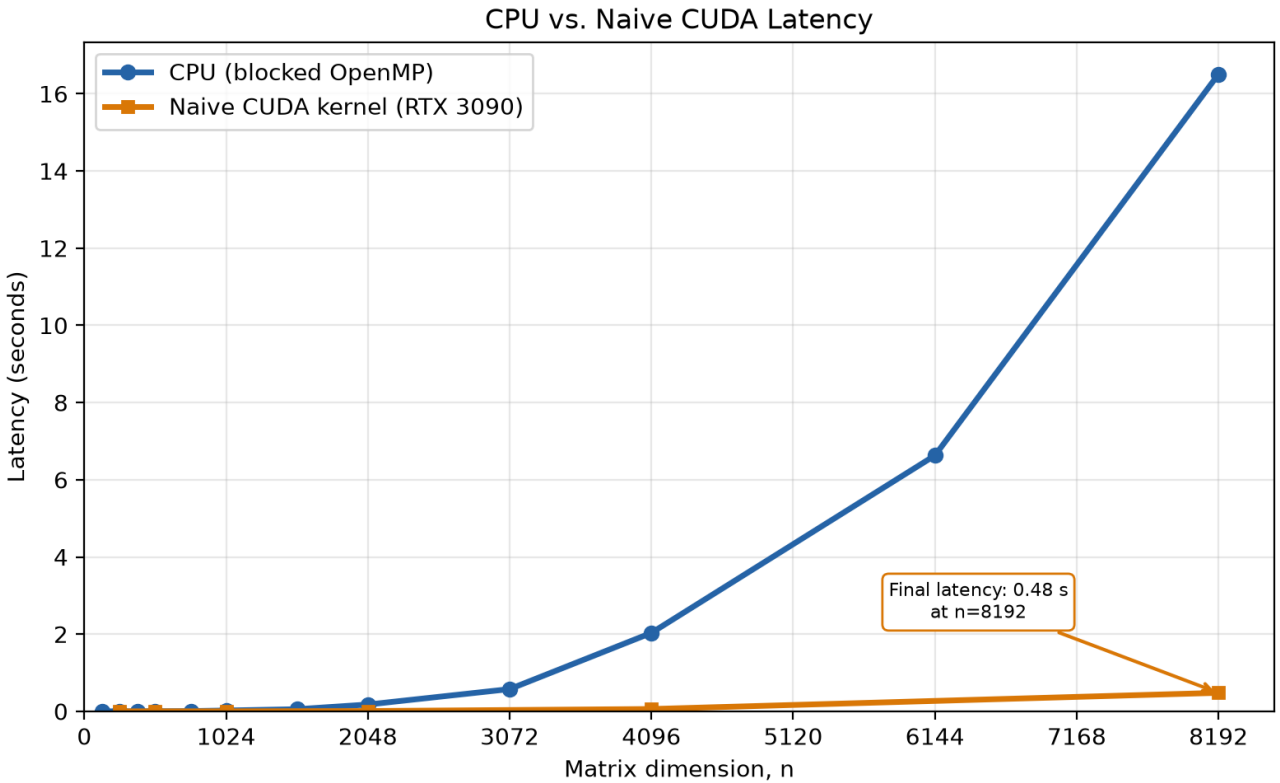
Example (One thread per output element)

We want to do a plain matrix multiplication between matrices A and B and store it into C. Let A have shape 5000×3000, and B have shape 3000×4000. Thus, C must have shape 5000×4000. $m = 5000$, $n = 4000$, $k = 3000$.

We want blocks of 32×32, so $BM, BN = 32$.

```
launch_sgemm_naive<32,32>(M, N, K, alpha, dA, dB, beta, dC);
// inside launcher function
  dim3 blockDim(32, 32);
  dim3 gridDim(126, 158); // Ceil division
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C)
```

Naive CUDA is already faster than the CPU

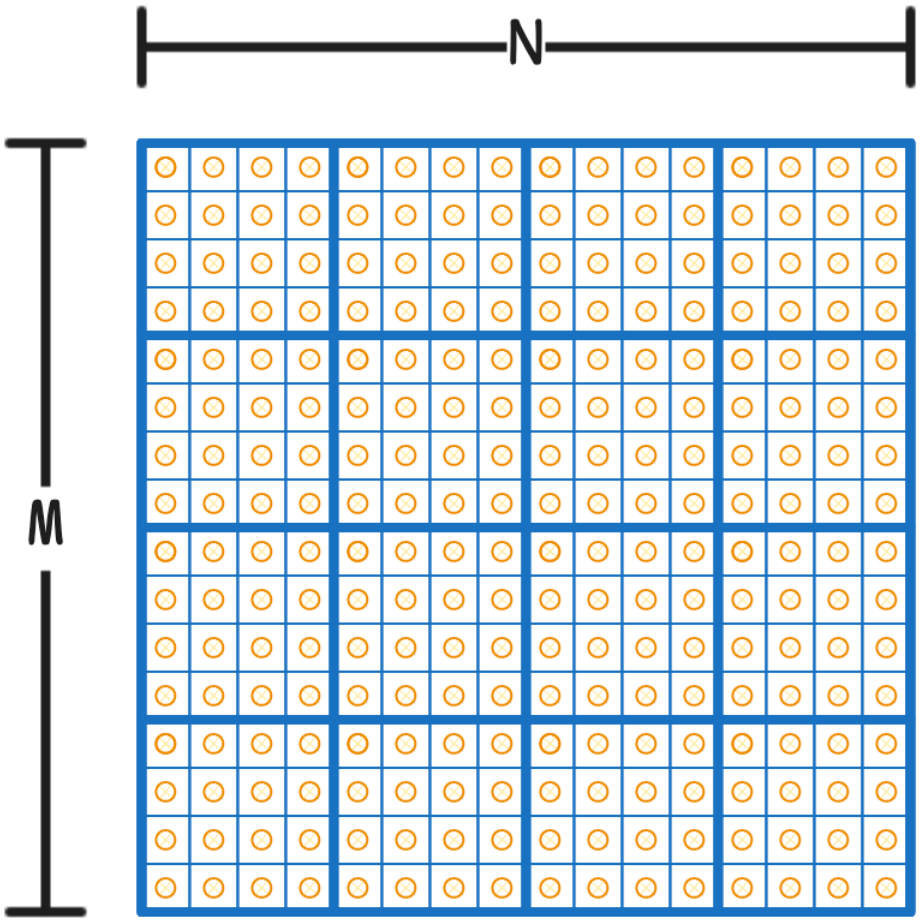


Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

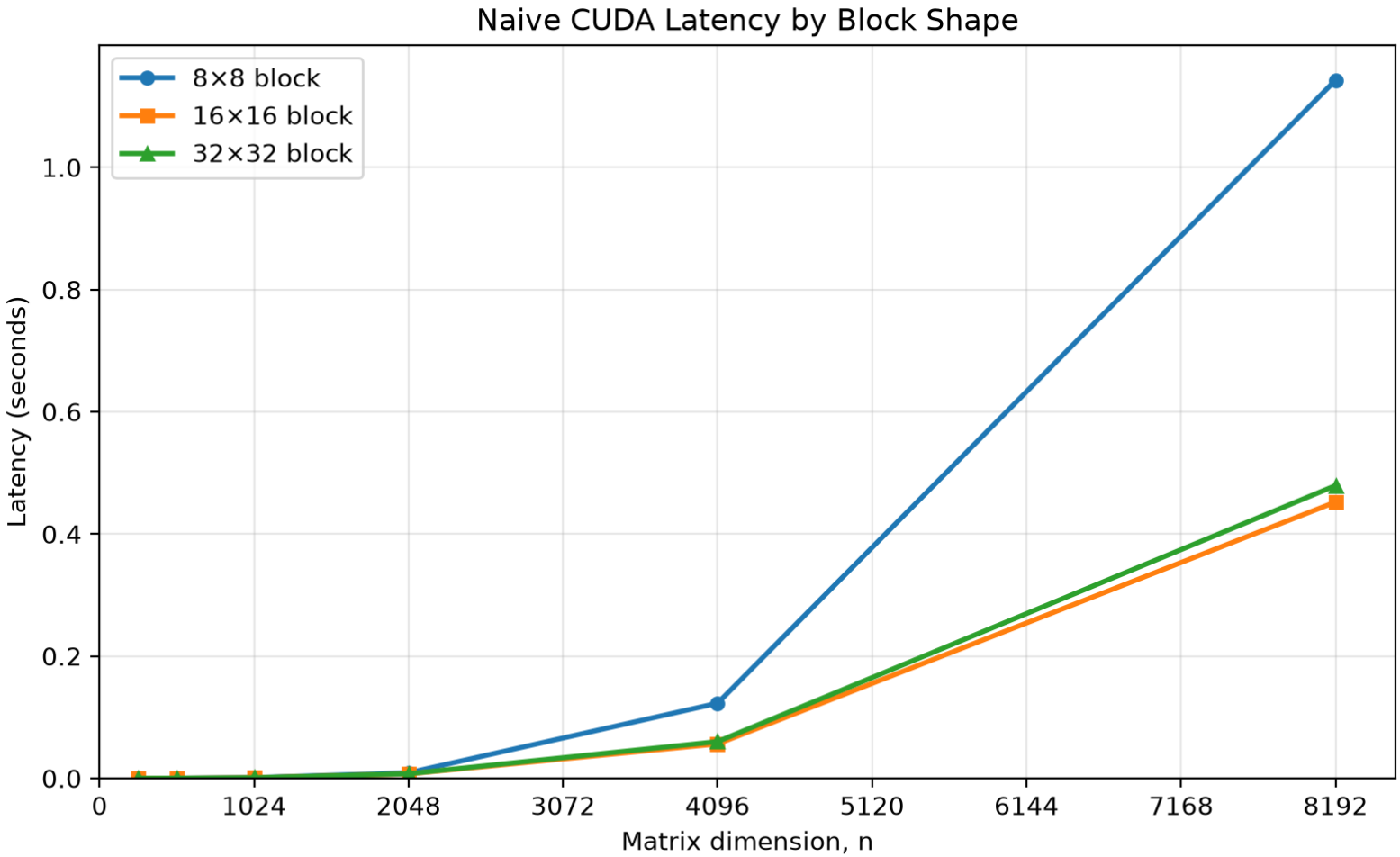
Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



Let's do some experimenting

This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?



Shared Memory (SMEM) Caching

Programmable L1 Cache (SMEM)

Remember how our launch config required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so **threads in the same block have access to the SM's L1 cache/SMEM.**

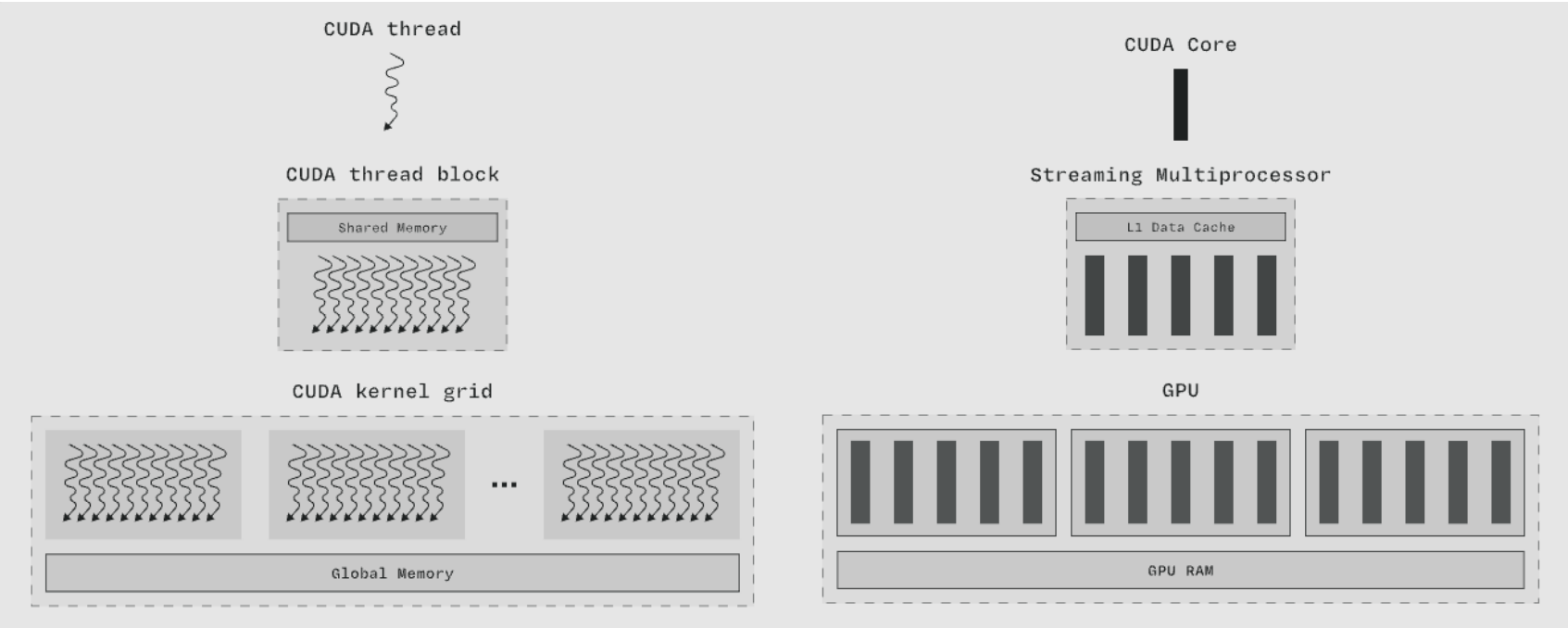


Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's.
The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's.
The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

Block shape	**L1 Cache Hit Rate**
8×8	83.51%
16×16	87.13%
32×32	94.98%

RTX 3090, $M = N = K = 4096$

Blocks and SMs

Why does block size matter?

This happens because entire **blocks** are **arbitrarily** scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

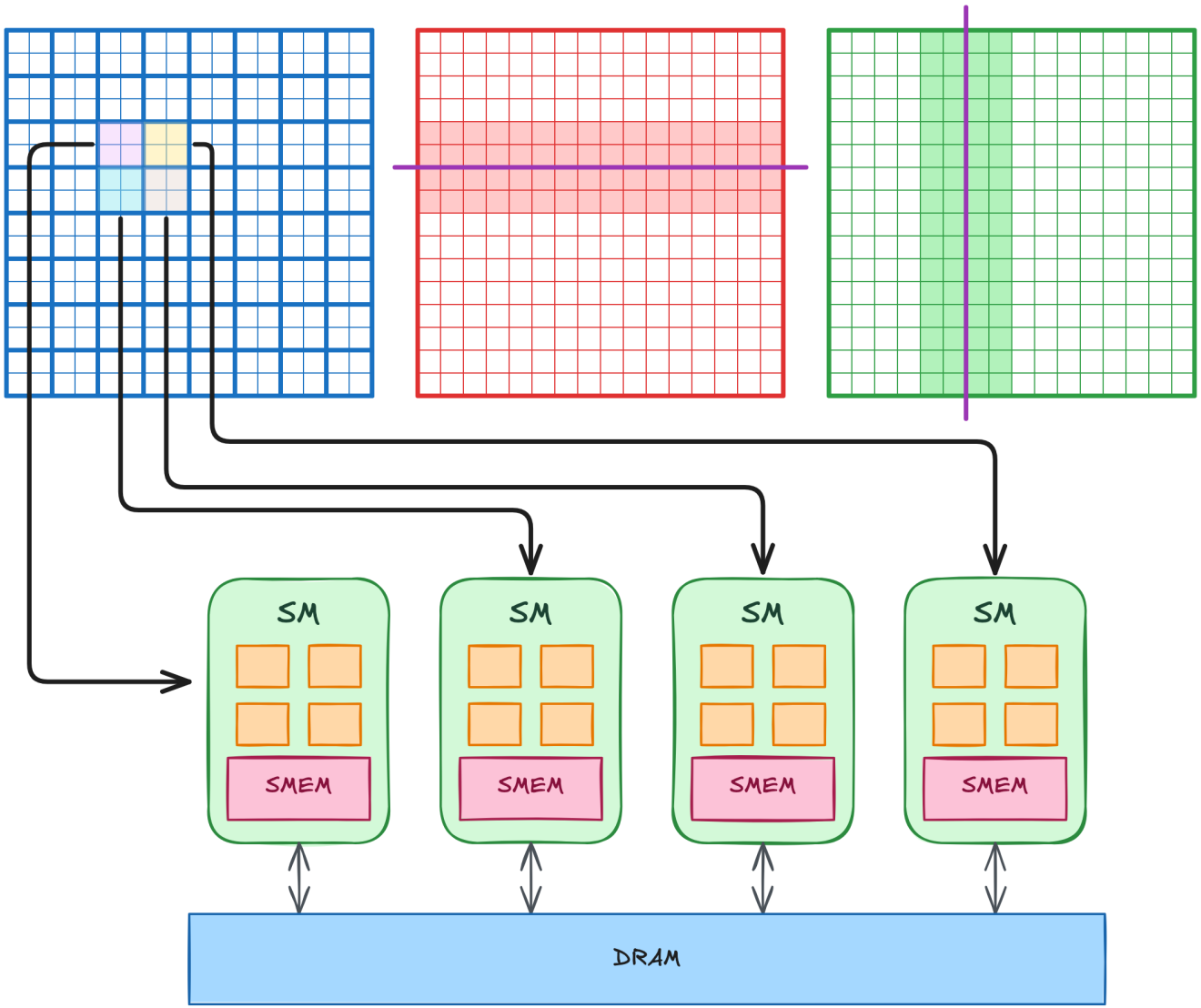
Block shape	**L1 Cache Hit Rate**
8×8	83.51%
16×16	87.13%
32×32	94.98%

RTX 3090, $M = N = K = 4096$

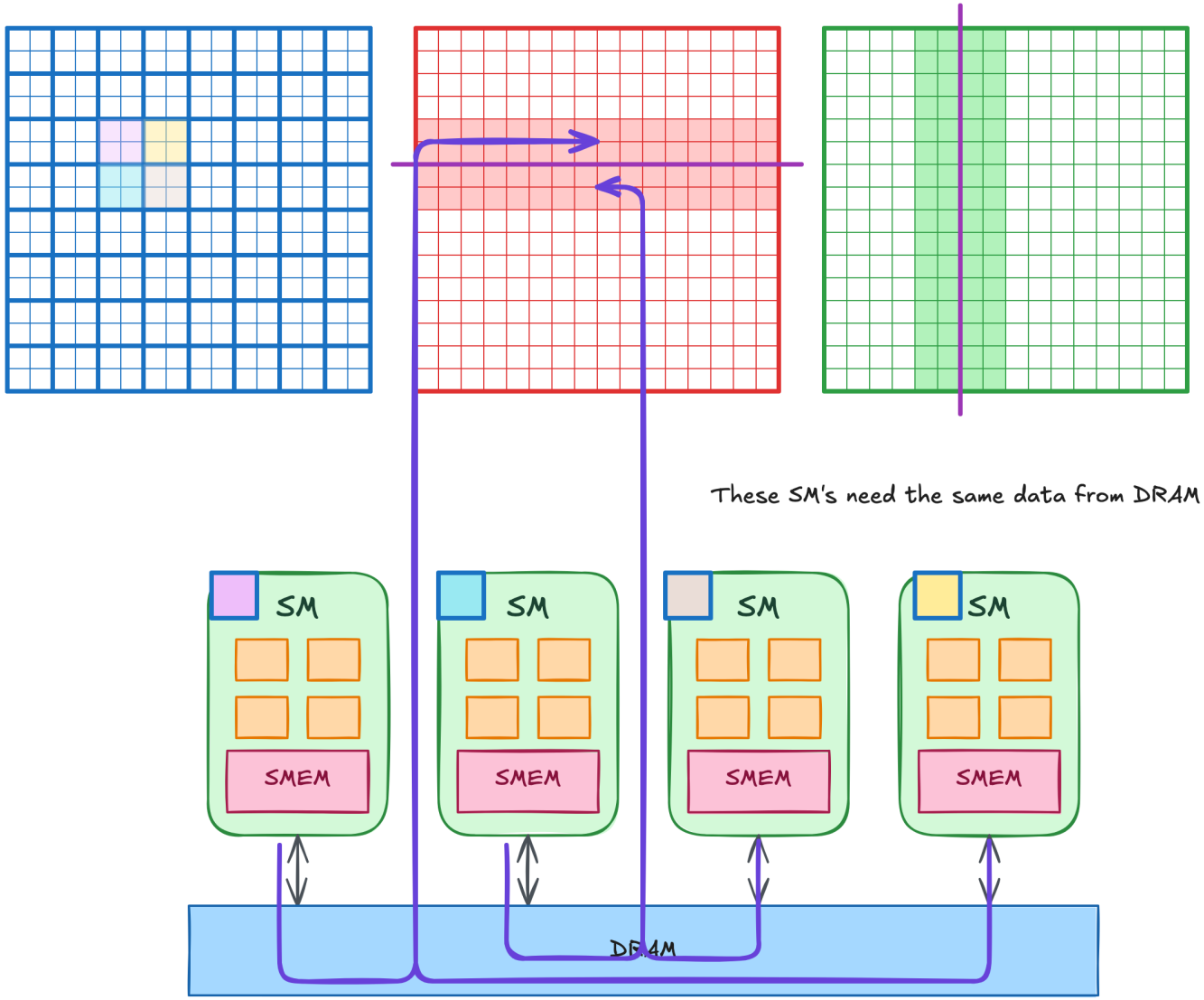
Intuition

Nearby elements of C require **nearby elements in** A **and** B. Blocks are assigned to SM's. Therefore, larger blocks mean that larger patches of C are assigned the to same SM's, increasing the chances of L1 cache hits.

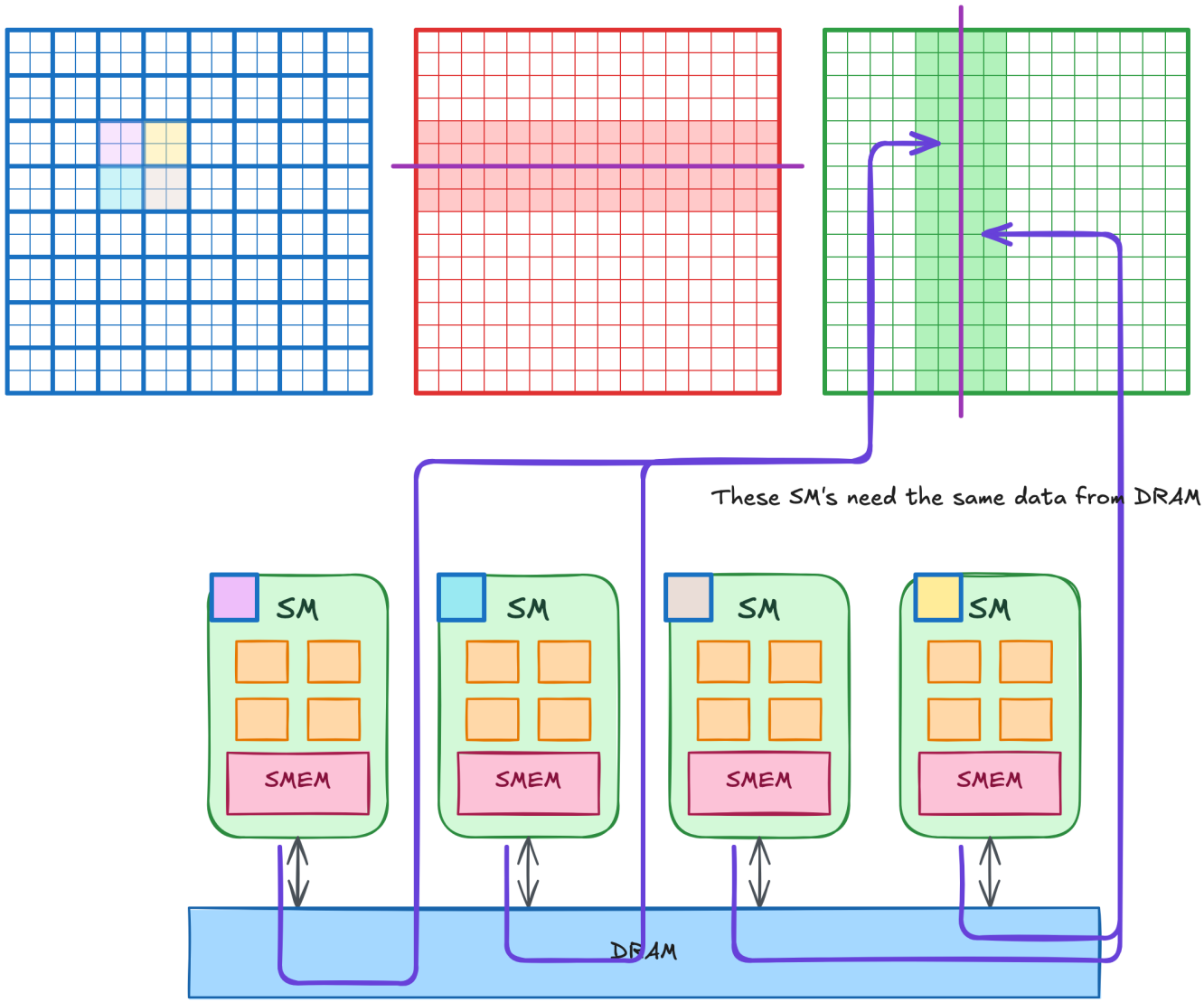
Why Larger Blocks Improve Cache Reuse



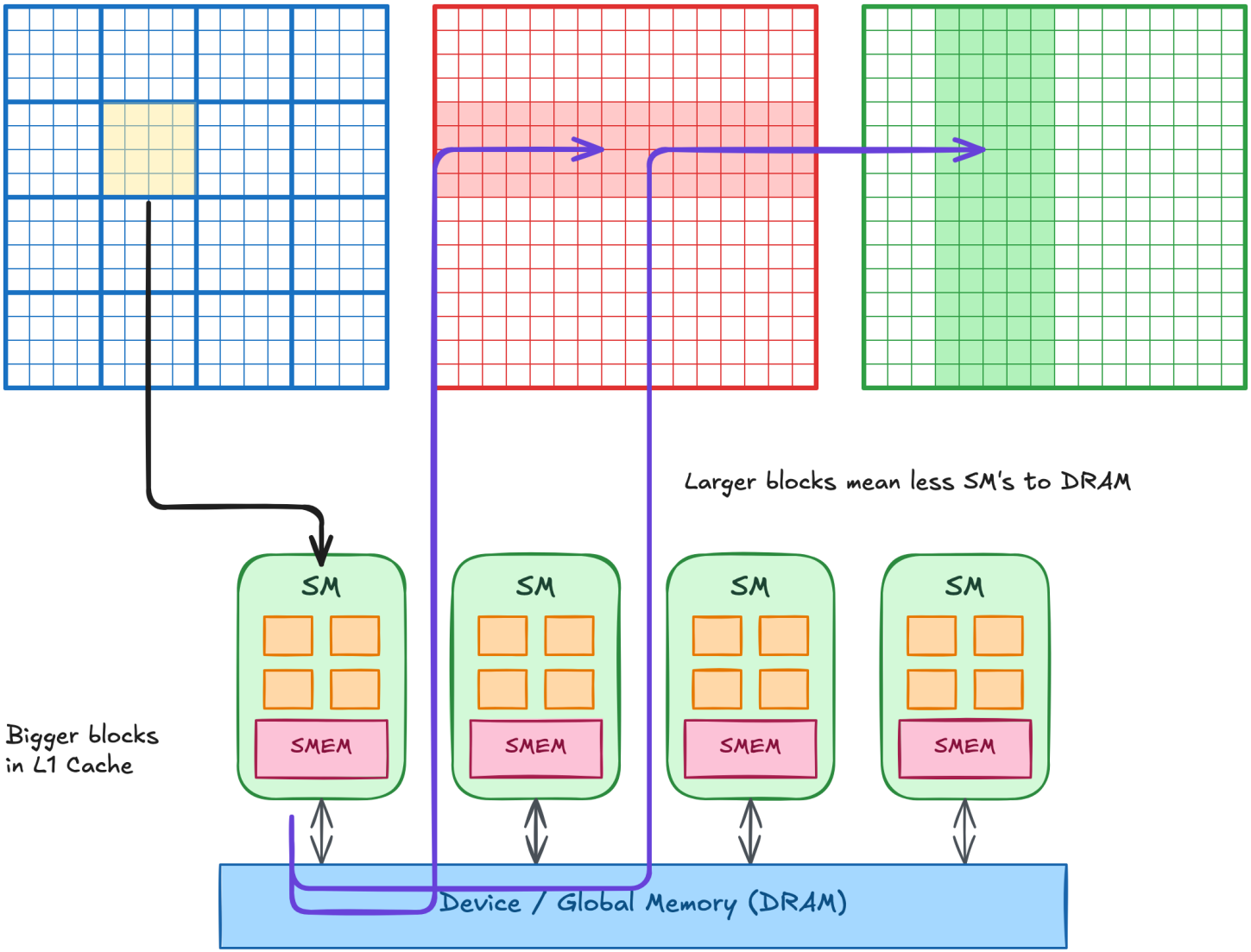
Why Larger Blocks Improve Cache Reuse



Why Larger Blocks Improve Cache Reuse



Why Larger Blocks Improve Cache Reuse



Tiling: Explicit Cache use

Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

Basic tiled SGEMM: step by step

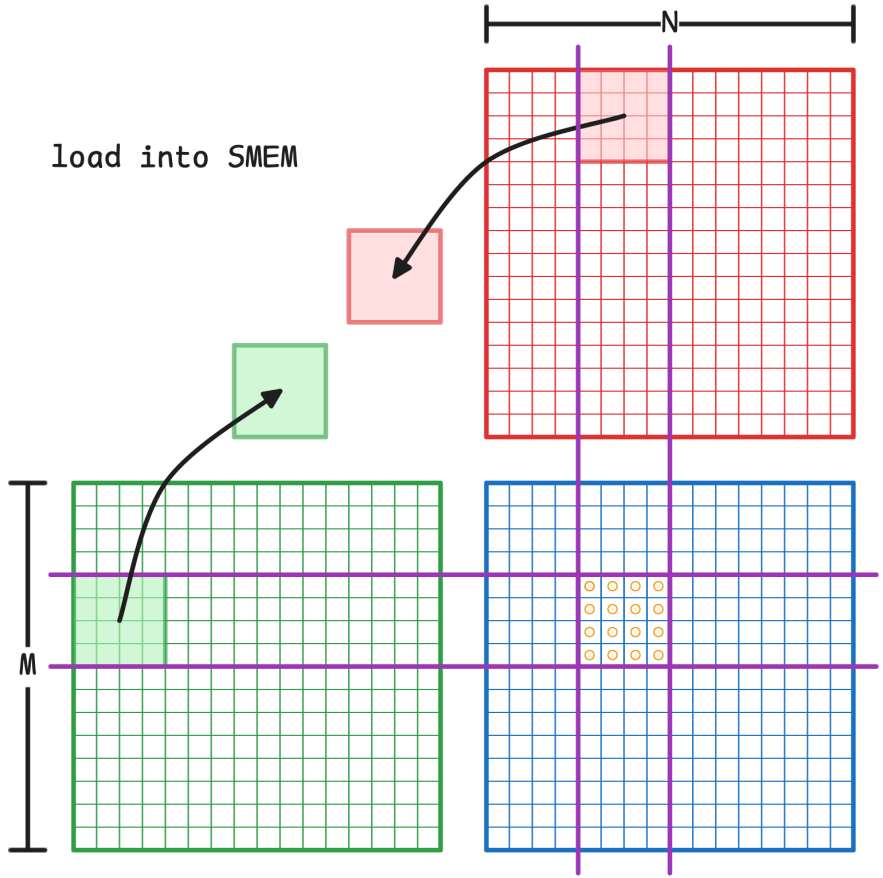
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

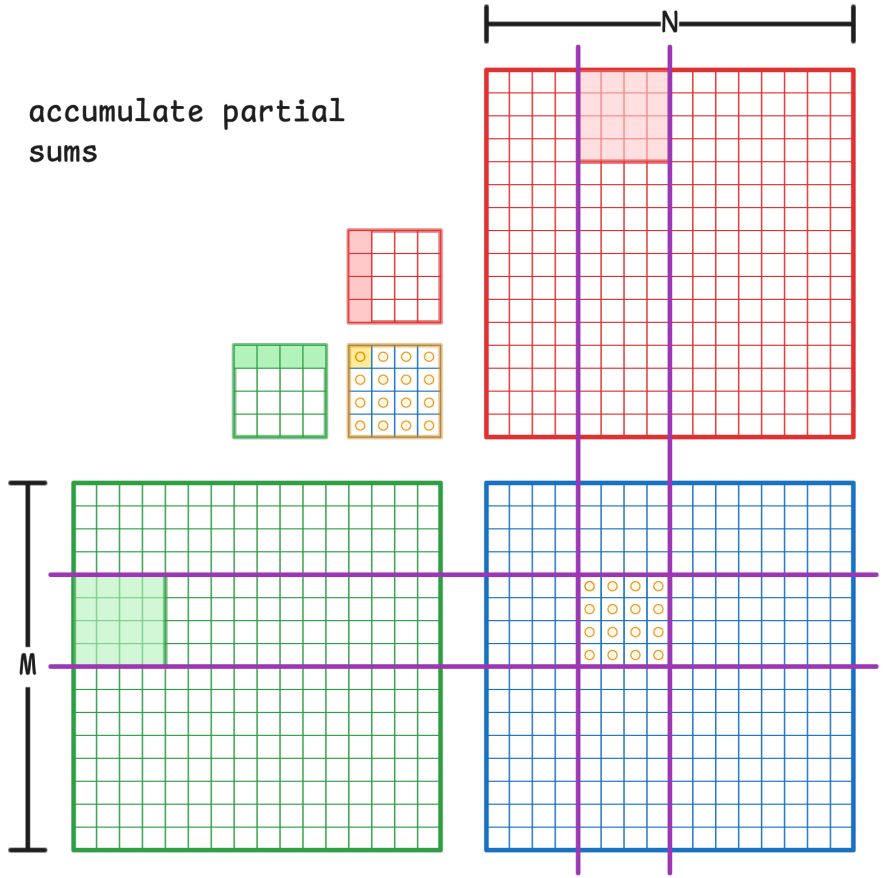
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

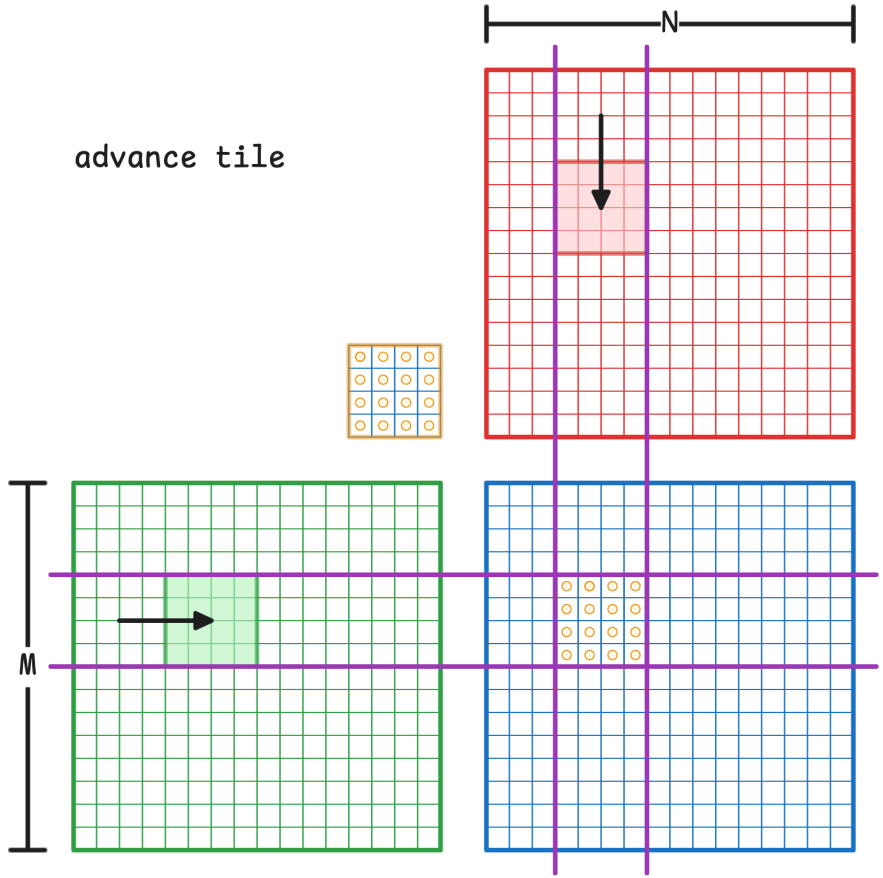
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

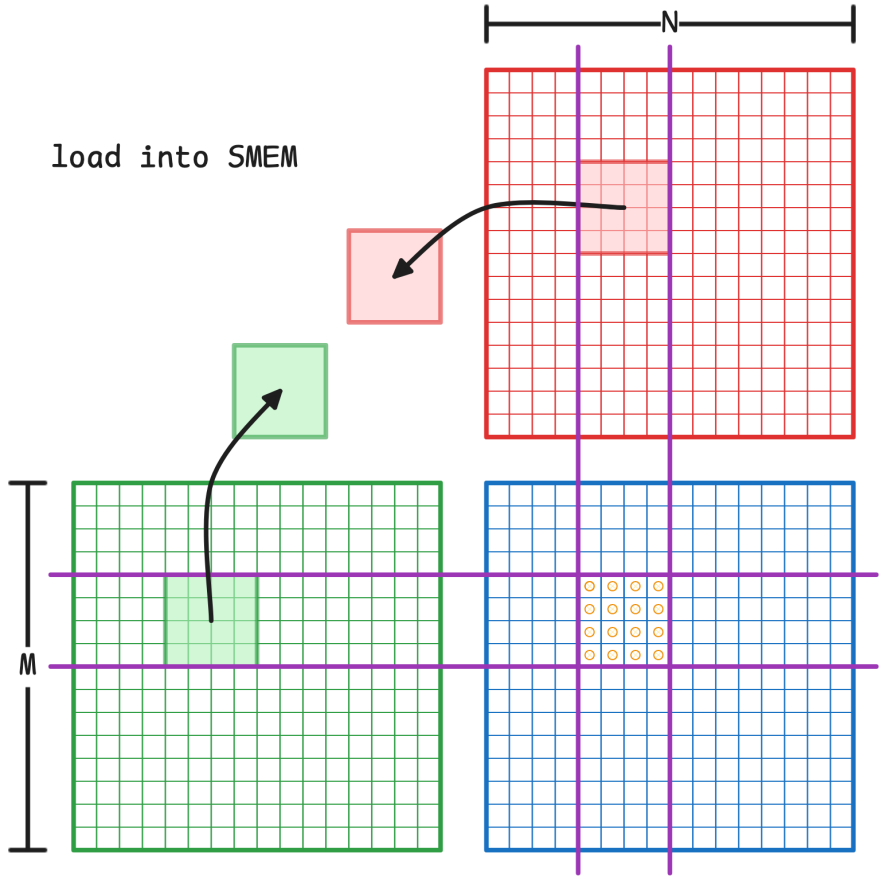
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

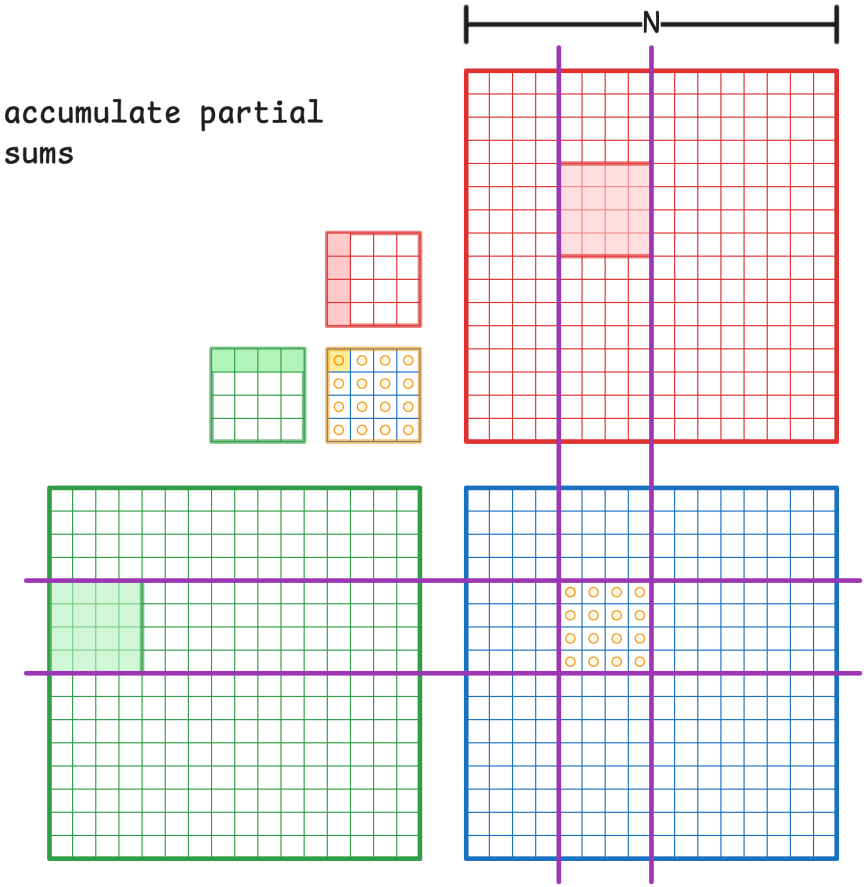
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```



Basic tiled SGEMM: step by step

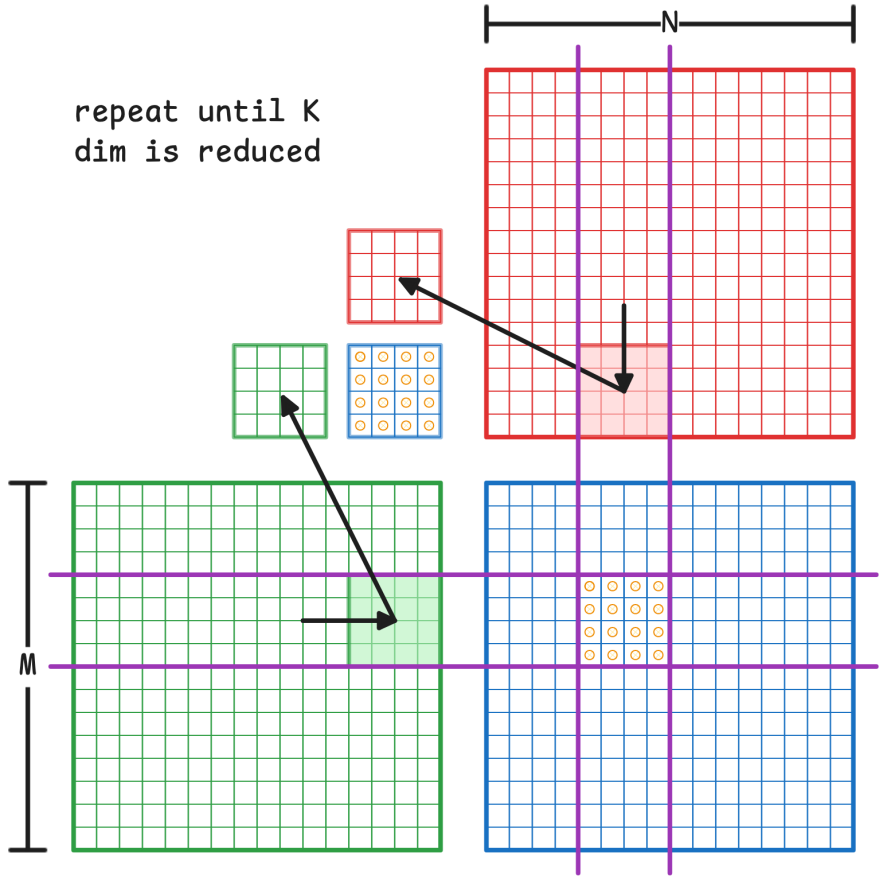
```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];
float sum = 0.0;

for (int k = 0; k < K; k += BK) {

  const int A_tile_x = k + xt;
  if (y < M && A_tile_x < K) {
    As[yt][xt] = A[y * K + A_tile_x];
  } else {
    As[yt][xt] = 0.0;
  }

  const int B_tile_y = k + yt;
  if (B_tile_y < K && x < N) {
    Bs[yt][xt] = B[B_tile_y * N + x];
  } else {
    Bs[yt][xt] = 0.0;
  }
  __syncthreads();

  for (int i = 0; i < BK; i++) {
    sum += As[yt][i] * Bs[i][xt];
  }
  __syncthreads();
}
```

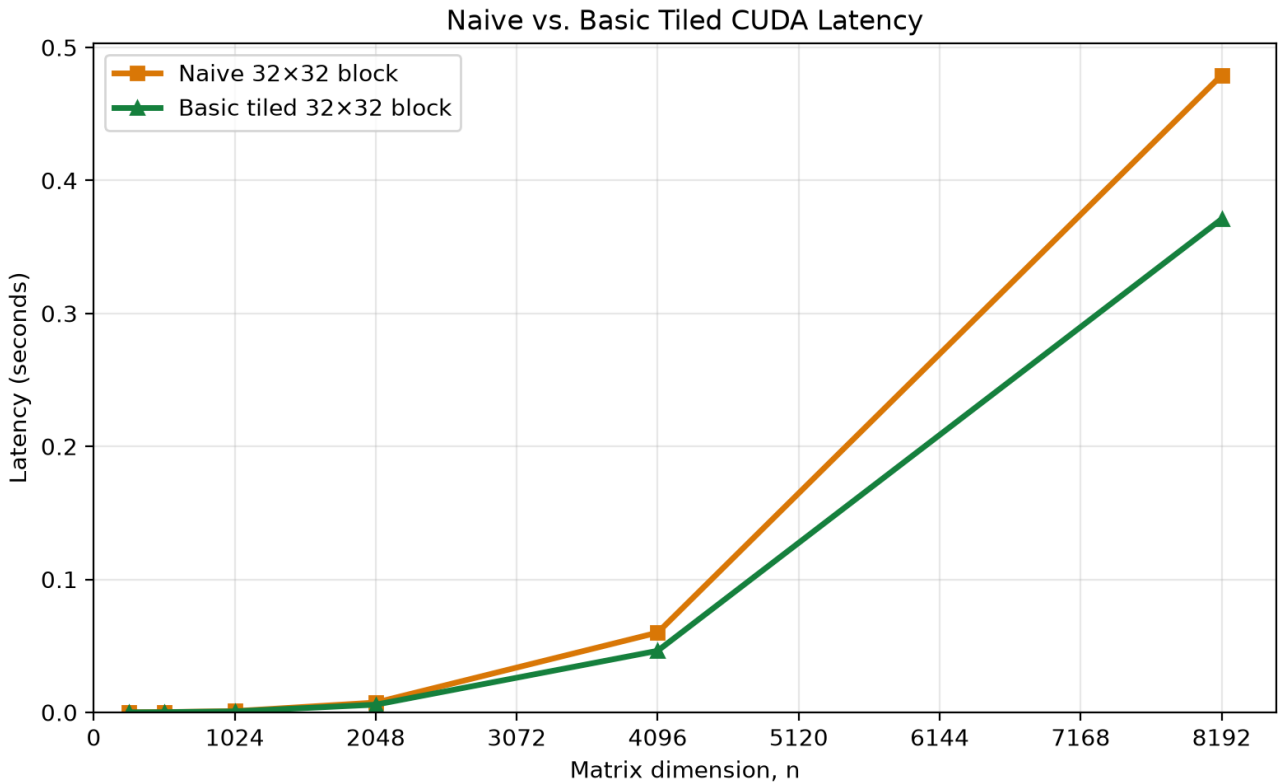


Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Tiling performance



At $n, m, k = 8192$, tiled 32×32 latency is 371 ms, a 22.6% improvement over the naive.

Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

Driving Question

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Driving Question

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Is there a better performance metric?

We have been measuring our kernels with **latency**, but is there some other metric that better describes "performance"?

Driving Question

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Is there a better performance metric?

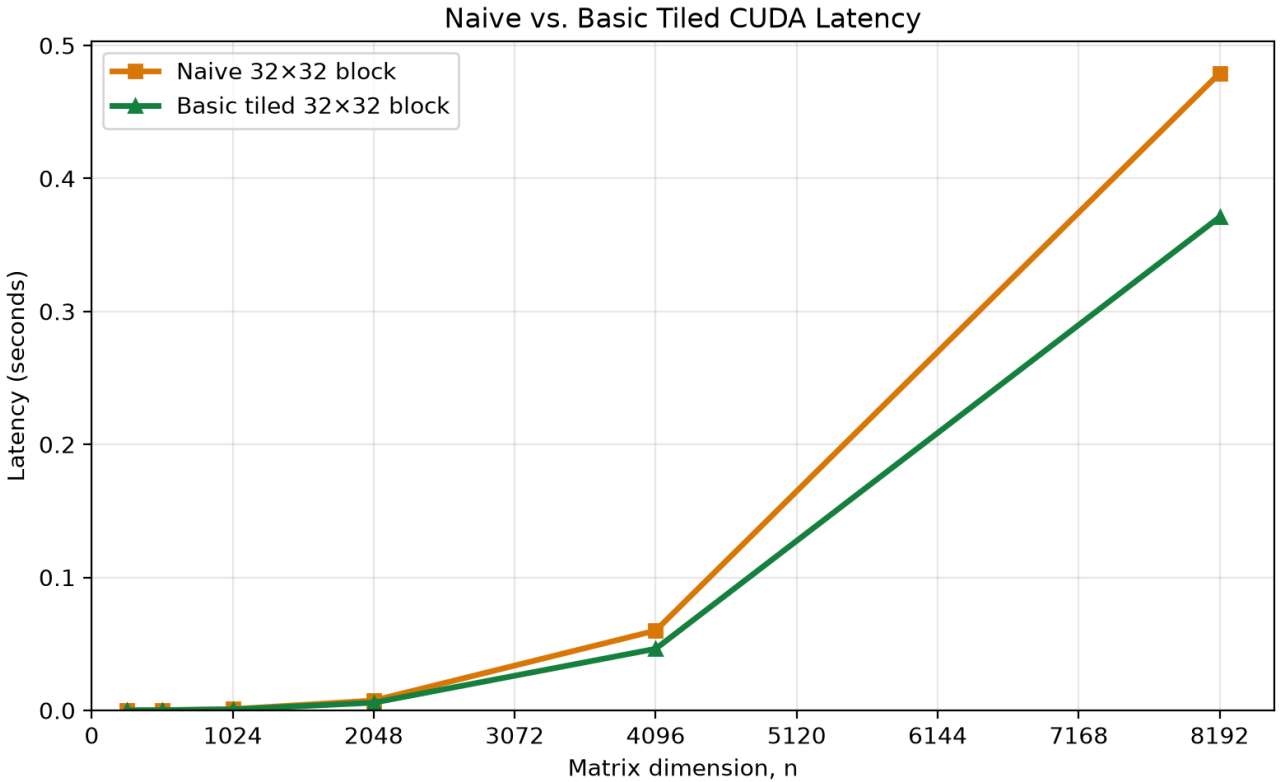
We have been measuring our kernels with **latency**, but is there some other metric that better describes "performance"?

How fast *can* we be?

Given our hardware constraints, how fast (theoretically) can we do our SGEMM operations?

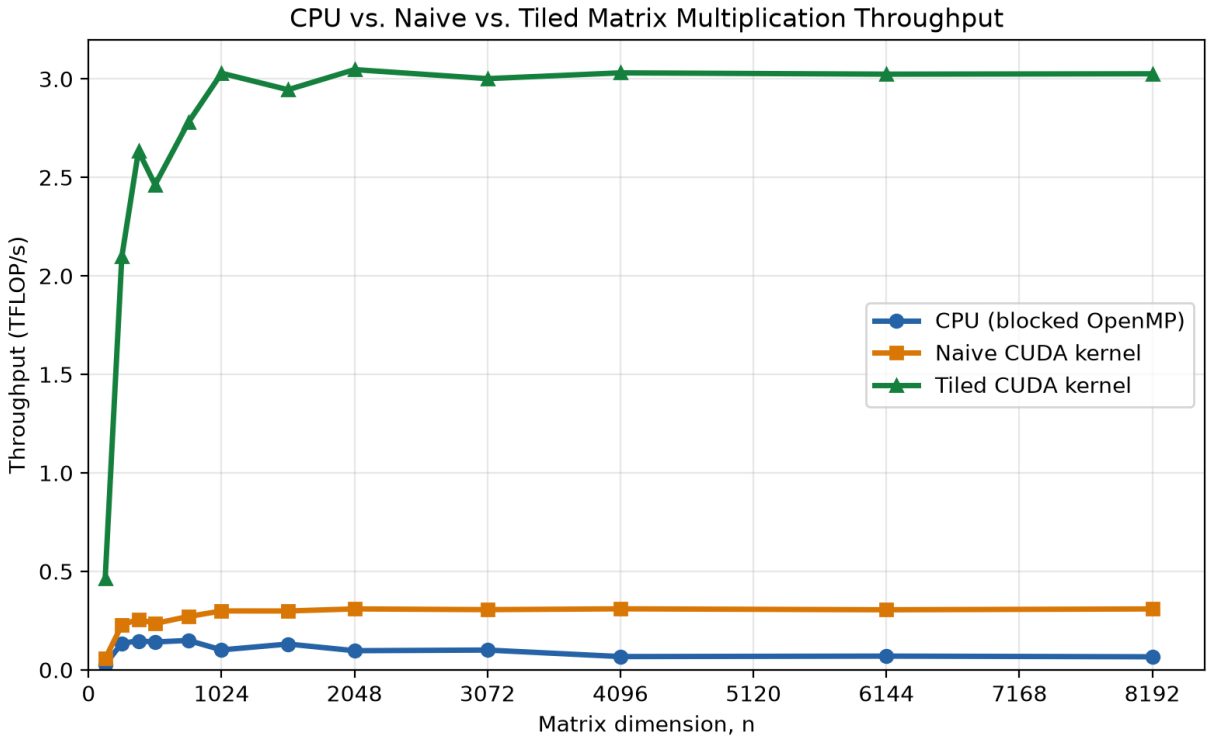
Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.



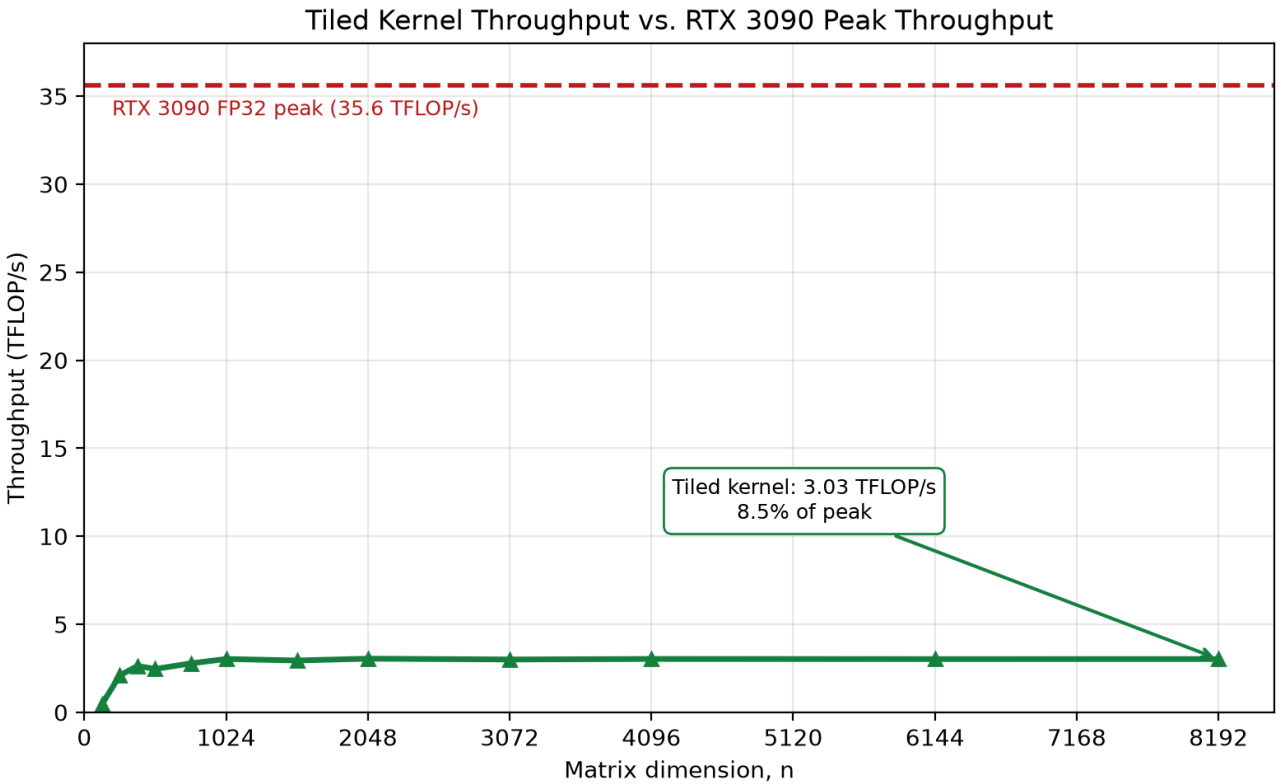
Why latency alone isn't enough

- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.



Why latency alone isn't enough

- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.



Why latency alone isn't enough

- Latency tells us how long one kernel launch takes, but it depends on the problem size.
- To compare performance **across problem sizes**, we need to account for both the work completed and the time required. This gives us **achieved throughput**.
- Since hardware is characterized by its **peak throughput**, the two values provide a first gauge of how much compute capability our kernel realizes.

Why throughput

Throughput tells us

1. A general performance metric, no matter the problem size
2. How well we are utilizing hardware

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time.

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Throughput as a performance measure

Throughput

Throughput is the total useful work (FLOPs) done per unit time.

$$\text{Throughput} = \frac{\text{FLOPs}}{\text{time}}$$

Example

Peak throughput of our RTX 3090 is 35.6 TFLOP/s. We observed 360ms or 0.36s of latency for $n = 8192$, where the total FLOPs required is 1.10 TFLOPs. Thus, we have an observed throughput of

$$\frac{1.10\text{TFLOPs}}{0.36\text{s}} \approx 3.05\,\text{TFLOP/s}.$$

Roughly 8.5% of peak throughput.

Characterizing our kernels data use

RTX 3090		
Peak throughput	35.6	TFLOP/s
DRAM bandwidth	935.8	GB/s

1. Throughput characterizes how well our kernel utilizes the **compute hardware**.
2. One factor that impacts final throughput is how efficiently our kernels use data.

How can we quantify how "well" our kernel uses data?

We observed that we can improve the performance of our SGEMM kernel with tiling, an optimization that improves *data use*.

Characterizing our kernels data use

RTX 3090		
Peak throughput	35.6	TFLOP/s
DRAM bandwidth	935.8	GB/s

1. Throughput characterizes how well our kernel utilizes the **compute hardware**.
2. One factor that impacts final throughput is how efficiently our kernels use data.

How can we quantify how "well" our kernel uses data?

We observed that we can improve the performance of our SGEMM kernel with tiling, an optimization that improves *data use*. **Can we quantify this data usage?**

Arithmetic Intensity: Characterizing our kernels

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Arithmetic Intensity: Characterizing our kernels

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example (Naive kernel)

For $n, m, k = 8192$, in the worst case, the naive kernel loads (from DRAM)

$$n \times m \times 2k \times 4 \text{ bytes} \approx 4398 \text{ GB}.$$

We do 1.10 TFLOPs of work, so we have an AI of

$$\text{AI} = \frac{1.10 \text{ TFLOP}}{4.398 \text{ TB}} \approx 0.25 \text{ FLOP/byte}.$$

Arithmetic Intensity: Characterizing our kernels

Arithmetic Intensity

We define the **Arithmetic Intensity** (AI) of our kernel as the number of FLOPs done per bytes moved from DRAM.

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}$$

Example (Tiled kernel)

For $n, k, m = 8192$, in the worst case, our tiled kernel loads (from DRAM)

$$\frac{m}{32} \times \frac{n}{32} \times 2\left(\frac{k}{32}\right) \times 32 \times 32 \times 4 \text{ bytes} = 137.4 \text{ GB}$$

With a total work count of 1.10 TFLOP, its arithmetic intensity is

$$\text{AI} = \frac{1.10 \text{ TFLOP}}{137.7 \text{ GB}} \approx 8.0 \text{ FLOP/byte}.$$

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

Example (Intuition)

In order for the CUDA Cores to do work, we have to first move the data from DRAM to the SM's.

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

Example (Intuition)

In order for the CUDA Cores to do work, we have to first move the data from DRAM to the SM's. If the kernel only does a small amount of work on the data (low AI), it will likely finish before the next batch of data arrives.

The Memory Roof

Memory Roof

Given the DRAM bandwidth and a kernel's AI, we say the **memory roof** of the kernel is

$$\text{Memory Roof} = \text{AI} \times \text{DRAM bandwidth}.$$

Example (Intuition)

In order for the CUDA Cores to do work, we have to first move the data from DRAM to the SM's. If the kernel only does a small amount of work on the data (low AI), it will likely finish before the next batch of data arrives. **In this case, the final measured throughput will be constrained by the DRAM bandwidth.** We are **memory bound.**

The Memory Roof

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At $935.8\,\mathrm{GB/s}$ of bandwidth, our **memory roof** is

$$935.8\,\mathrm{GB/s} \times 8.0\ \mathrm{FLOP/byte} \approx 7.49\,\mathrm{TFLOP/s}.$$

The Memory Roof

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At 935.8 $\mathrm{GB/s}$ of bandwidth, our **memory roof** is

$$935.8\,\mathrm{GB/s} \times 8.0\ \mathrm{FLOP/byte} \approx 7.49\,\mathrm{TFLOP/s}.$$

This is much lower than the peak throughput of 35.6 $\mathrm{TFLOP/s}$.

The Memory Roof

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Our tiling kernel has AI of 8.0 FLOP/byte, meaning it does 8 floating point operations for every byte of data moved from DRAM.
At $935.8\,\mathrm{GB/s}$ of bandwidth, our **memory roof** is

$$935.8\,\mathrm{GB/s} \times 8.0\ \mathrm{FLOP/byte} \approx 7.49\,\mathrm{TFLOP/s}.$$

This is much lower than the peak throughput of $35.6\,\mathrm{TFLOP/s}$. **No matter how we optimize this kernel, without increasing AI, we will never achieve greater than** $7.49\,\mathrm{TFLOP/s}$.

Hypothetical Kernel

RTX 3090		
Peak throughput	35.6	TFLOP/s
DRAM bandwidth	935.8	GB/s

Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s.

Hypothetical Kernel

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our memory roof is now

$$935.8\,\mathrm{GB/s} \times 16.0\ \mathrm{FLOP/byte} \approx 15.0\,\mathrm{TFLOP/s}.$$

Hypothetical Kernel

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

Example

Imagine we have some kernel that has double the AI at 16.0 FLOP/s. Our memory roof is now

$$935.8\,\mathrm{GB/s} \times 16.0\ \mathrm{FLOP/byte} \approx 15.0\,\mathrm{TFLOP/s}.$$

Since $15.0\,\mathrm{TFLOP/s} < 35.6\,\mathrm{TFLOP/s}$, this kernel will still be memory bound.

Hypothetical Kernels

RTX 3090		
Peak throughput	35.6	TFLOP/s
DRAM bandwidth	935.8	GB/s

AI (FLOP/byte)	Memory Roof (TFLOP/s)
8	7.49
16	14.97
32	29.95
36	33.69
38	35.56
40	37.43

Hypothetical Kernels

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

AI (FLOP/byte)	Memory Roof ($\mathrm{TFLOP/s}$)
8	7.49
16	14.97
32	29.95
36	33.69
38	35.56
40	37.43

Wait! Our peak throughput is 35.6 $\mathrm{TFLOP/s}$! At AI ≥ 38, we are **compute bound**. Our data can finally keep up with the CUDA Cores!

Ridge Point

RTX 3090		
Peak throughput	35.6	$\mathrm{TFLOP/s}$
DRAM bandwidth	935.8	$\mathrm{GB/s}$

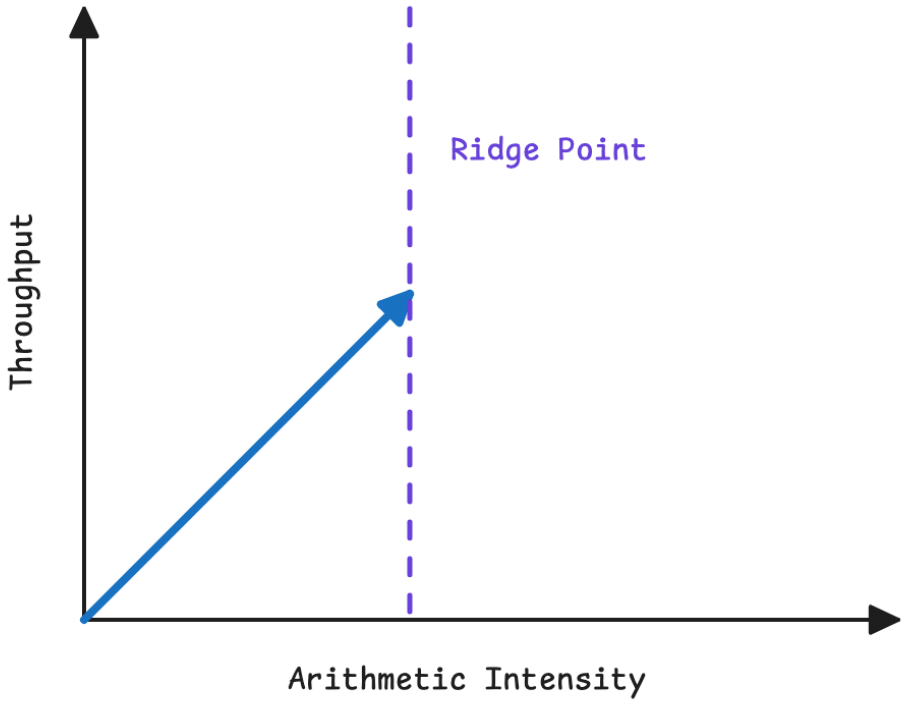
AI $= 38$ FLOP/byte is known as the **ridge point** *RP*, and can be found by solving:

$$\begin{aligned}
\text{Data roof} &= \text{Peak throughput} \\
\text{DRAM Bandwidth} \times \text{RP} &= \text{Peak throughput} \\
\text{RP} &= \frac{\text{Peak throughput}}{\text{DRAM Bandwidth}} \\
\text{RP} &= \frac{35.6\,\mathrm{TFLOP/s}}{935.8\,\mathrm{GB/s}} \approx 38.0 \text{ FLOP/s}
\end{aligned}$$

Roofline Model

Roofline Model

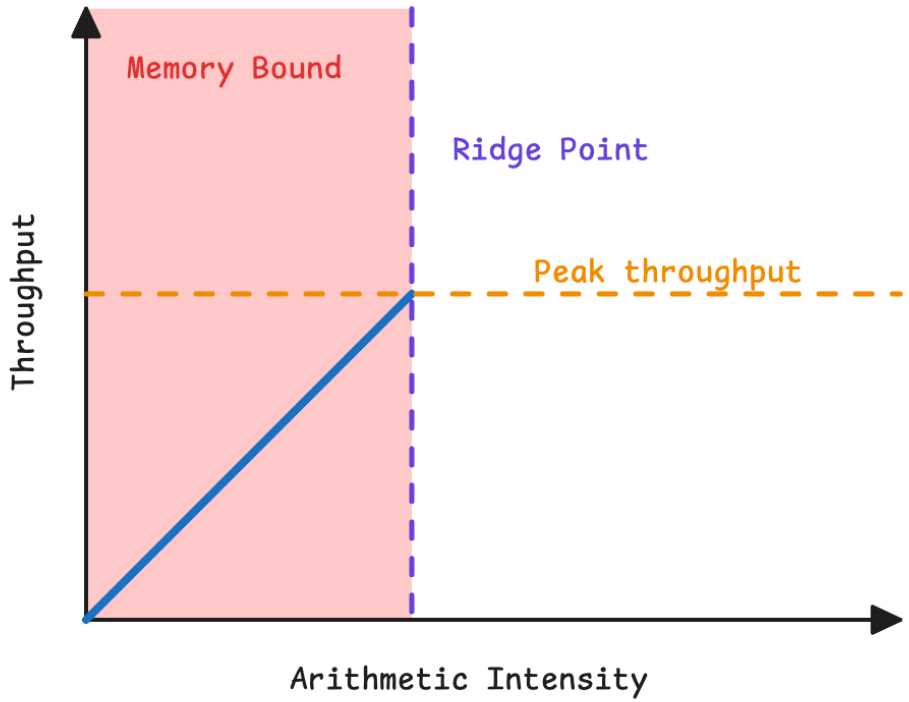
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI.



Roofline Model

Roofline Model

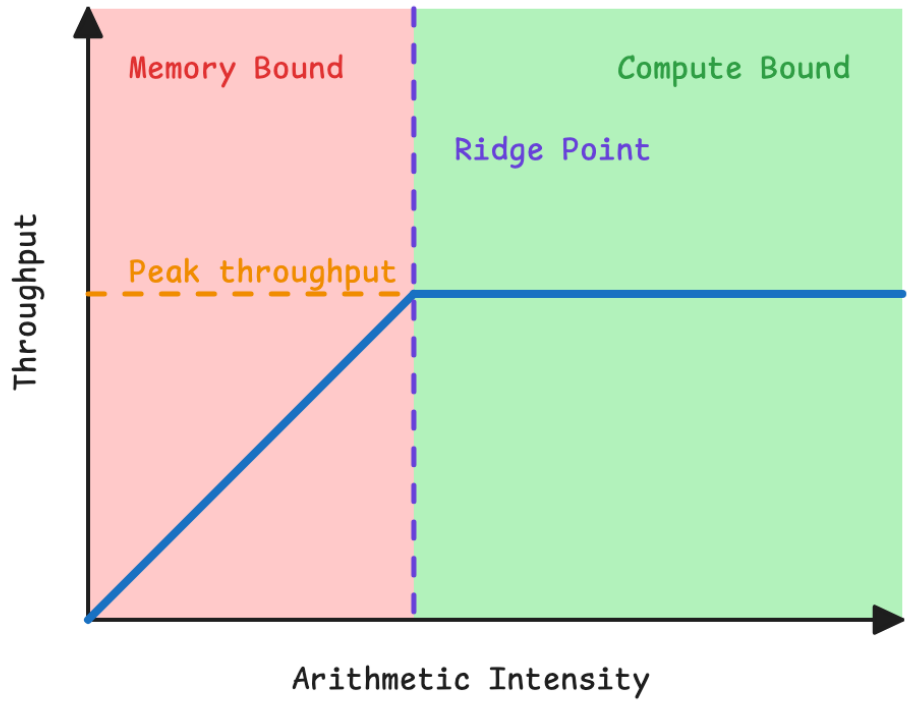
For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput.



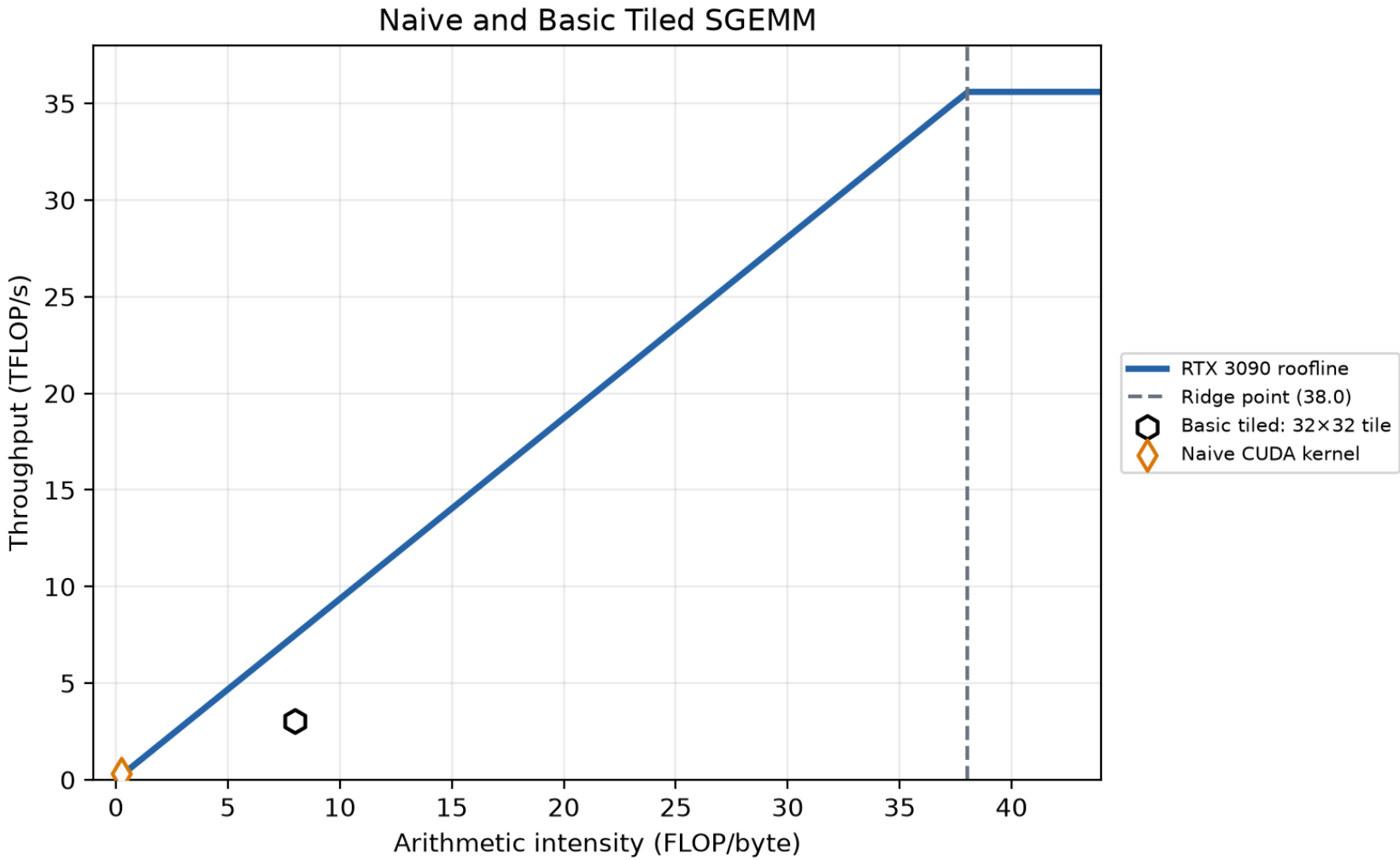
Roofline Model

Roofline Model

For a given system, we can predict a the **upper bound** of throughput of a kernel given its AI. Up to the **ridge point**, we are **memory bound**, increasing AI will raise the upper bound of throughput. Past that, we are **compute bound**, increasing AI will not raise the upper bound of throughput.



Roofline Model of our RTX 3090



Since we are memory bound, increasing AI should increase performance.

Section Overview

1. Optimizing **Ge**neral **M**atrix **M**ultiplication on CUDA
   - Introduction to CUDA, SGEMM, and the Naive Kernel
   - L1 Cache and Basic Tiling
   - Throughput, Arithmetic Intensity, and the Roofline Model
   - Applying the Roofline Model: Block Tiling and Thread Coarsening

Driving Question

How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.

But how can we actually increase AI?

Driving Question

How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Utilizing SMEM more aggressively.

Driving Question

How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Utilizing SMEM more aggressively.
2. Loading larger blocks into SMEM.

Driving Question

How can we incrase AI?

The roofline model tells us that increasing AI should increase throughput.
But how can we actually increase AI?

1. Utilizing SMEM more aggressively.
2. Loading larger blocks into SMEM.
3. Reusing values in SMEM to compute multiple values of C.

Generalized Tiling: Loading larger blocks.

Our tiled kernel **hardcoded** a square 32×32 block size. Each thread in the block load one value into SMEM. We can parameterize the tiling with BM, BN, BK.

Basic tiling: one thread, one element from each tile

```
__shared__ float As[BK][BK];
__shared__ float Bs[BK][BK];

for (int k = 0; k < K; k += BK) {
  const int Arow = y;
  const int Acol = k + xt;
  if (Arow < M && Acol < K)
    As[yt][xt] = A[Arow * K + Acol];
  else
    As[yt][xt] = 0.0;

  const int Brow = k + yt;
  const int Bcol = x;
  if (Brow < K && Bcol < N)
    Bs[yt][xt] = B[Brow * N + Bcol];
  else
    Bs[yt][xt] = 0.0;

  __syncthreads();
  // compute with As and Bs ...
}
```

Direct mapping

The thread coordinates directly select the shared-memory cells:

$$(y_t, x_t) \longrightarrow \begin{cases} A_s[y_t][x_t] \\ B_s[y_t][x_t] \end{cases}$$

Limitation

This only works because the block and both shared-memory tiles are all $BK \times BK$.

General tiling: threads cooperatively load each tile

```
const int tid = yt * BN + xt;
const int numThreads = BM * BN;

__shared__ float As[BM * BK];
__shared__ float Bs[BK * BN];
for (int k = 0; k < K; k += BK) {
  for (int id = tid; id < BM * BK;
       id += numThreads) {
    const int yid = id / BK;
    const int xid = id % BK;
    const int Arow = yb + yid;
    const int Acol = k + xid;
    As[id] = (Arow < M && Acol < K)
           ? A[Arow * K + Acol] : 0.0;
  }
  for (int id = tid; id < BK * BN;
       id += numThreads) {
    const int yid = id / BN;
    const int xid = id % BN;
    const int Brow = k + yid;
    const int Bcol = xb + xid;
    Bs[id] = (Brow < K && Bcol < N)
           ? B[Brow * N + Bcol] : 0.0;
  }
  __syncthreads();
  // Compute with As and Bs ...
}
```

Thread-strided loading

Flatten each tile. Thread *tid* loads

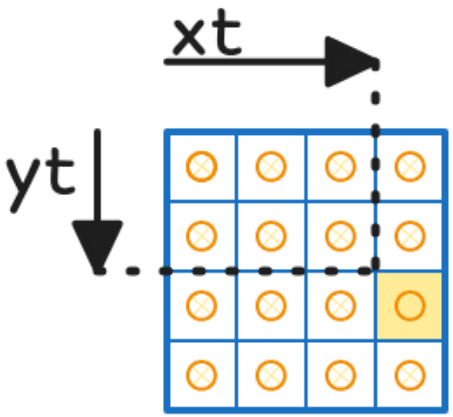
$$id = tid + jT$$

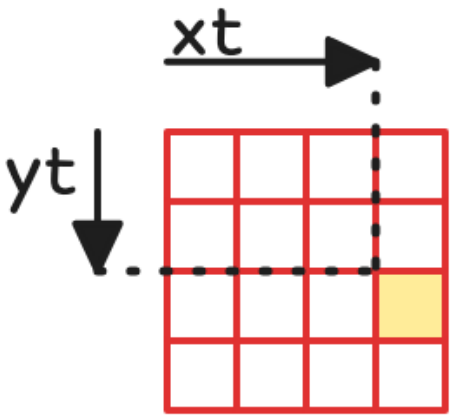
for $j = 0, 1, \ldots$, where $T = BM \cdot BN$ is the thread count.

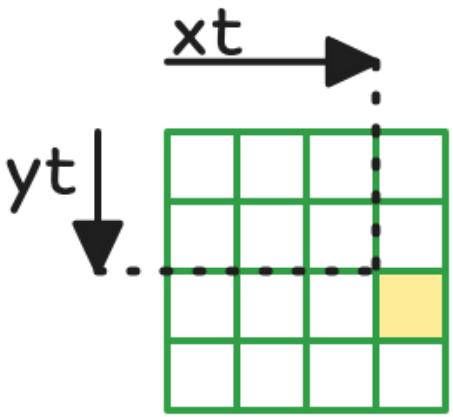
Key difference

A thread may load zero, one, or several values. Loading no longer assumes $BM = BN = BK$.

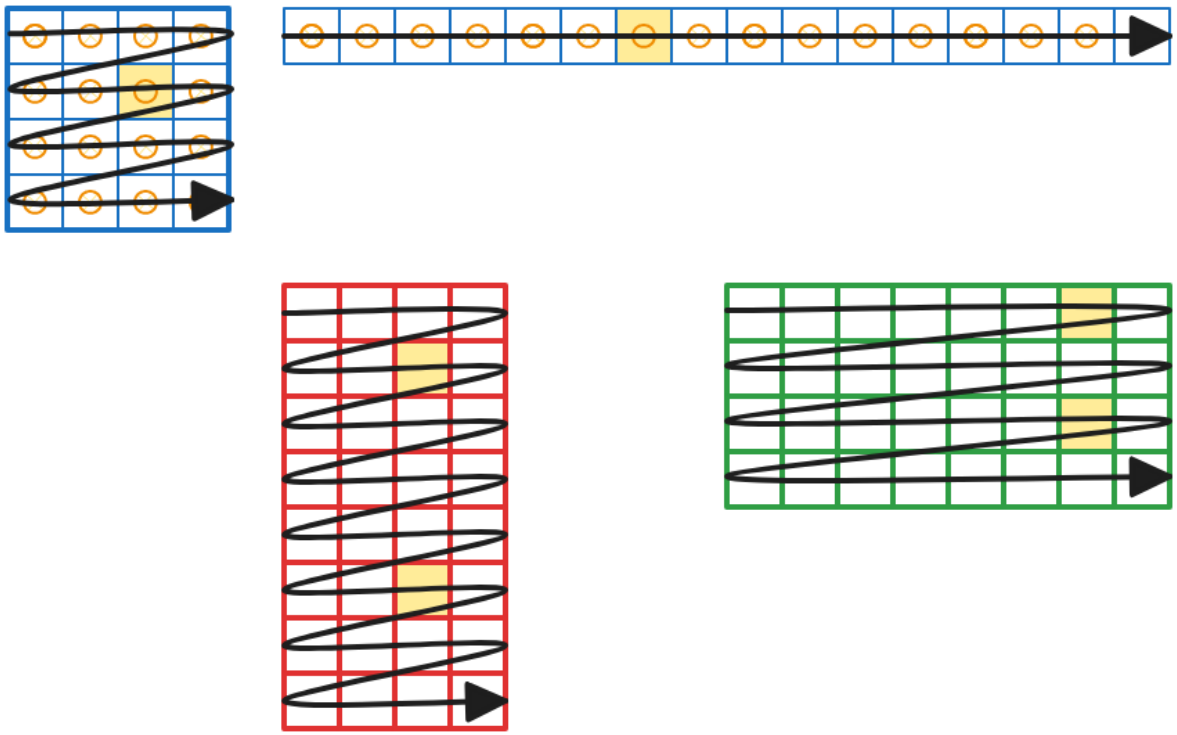
Visualization





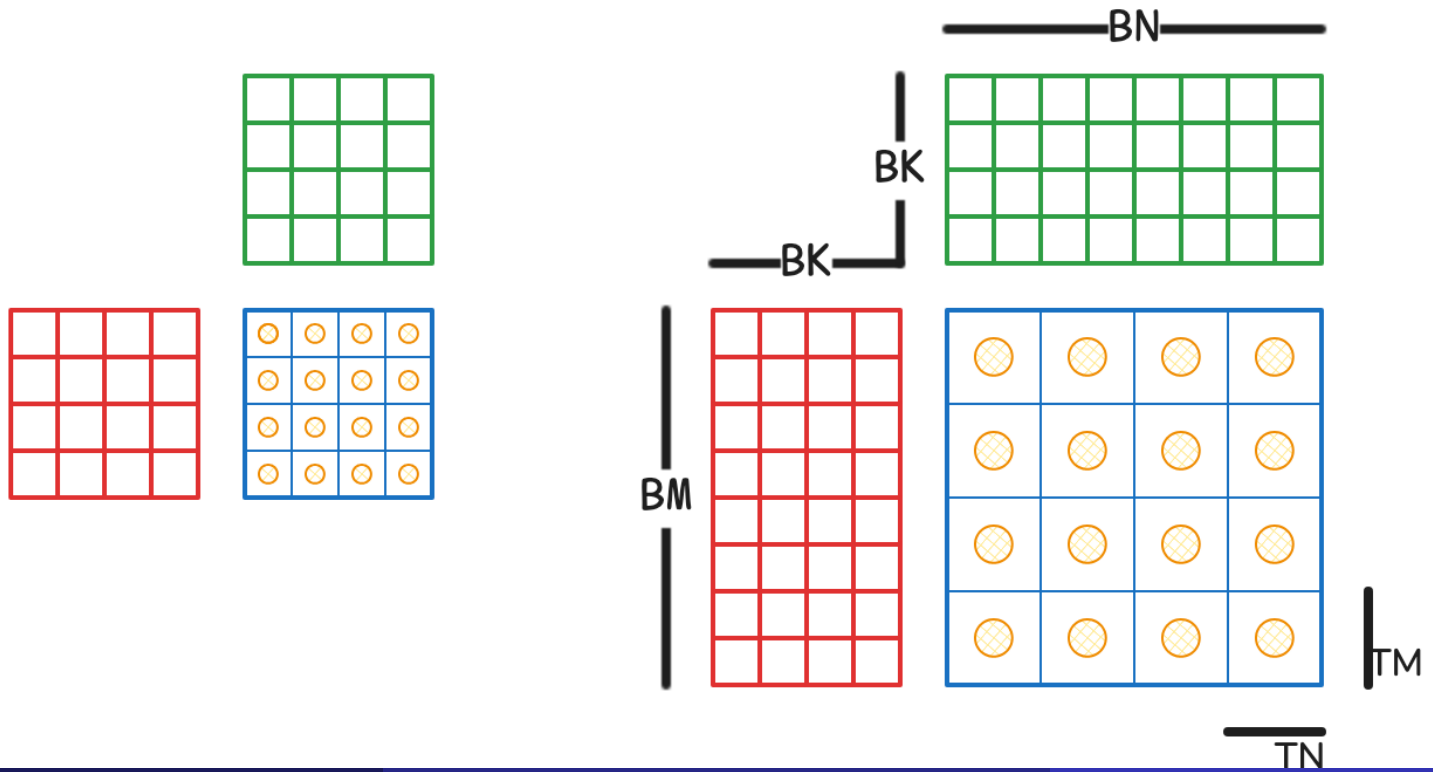


Visualization

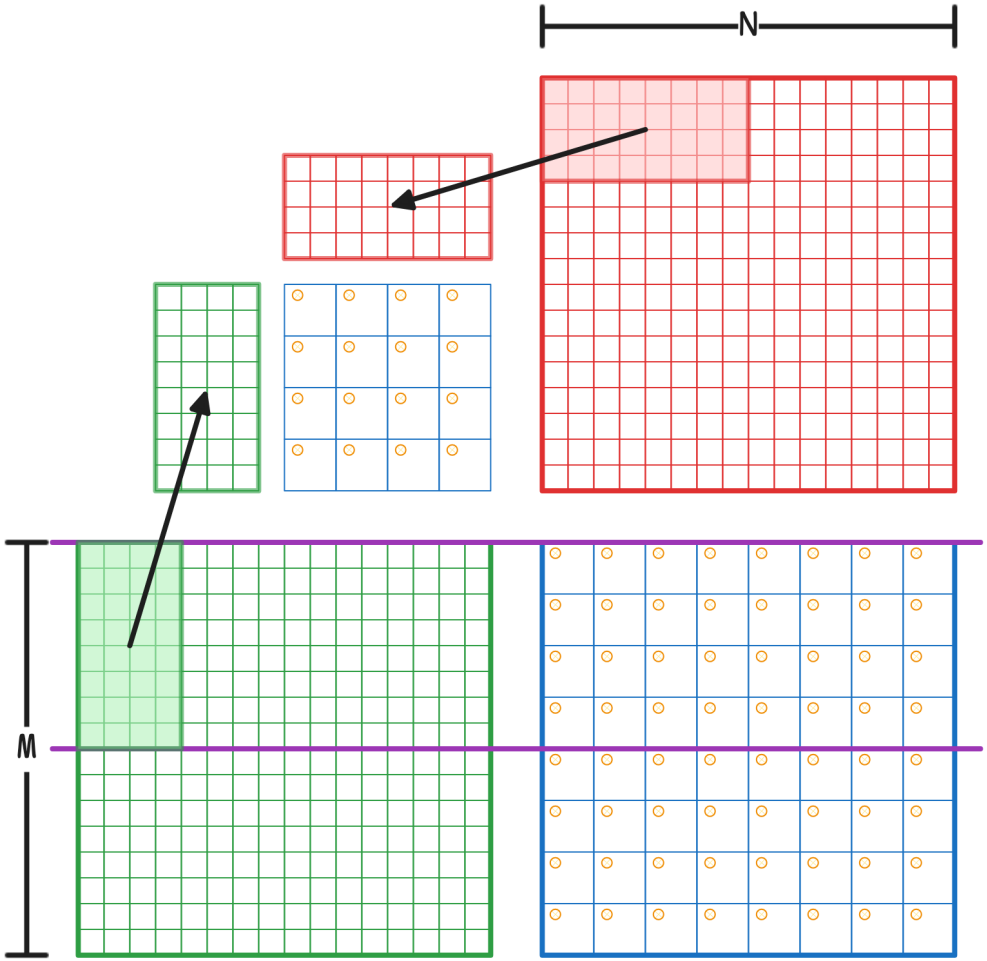


Thread coarsening: Increasing AI

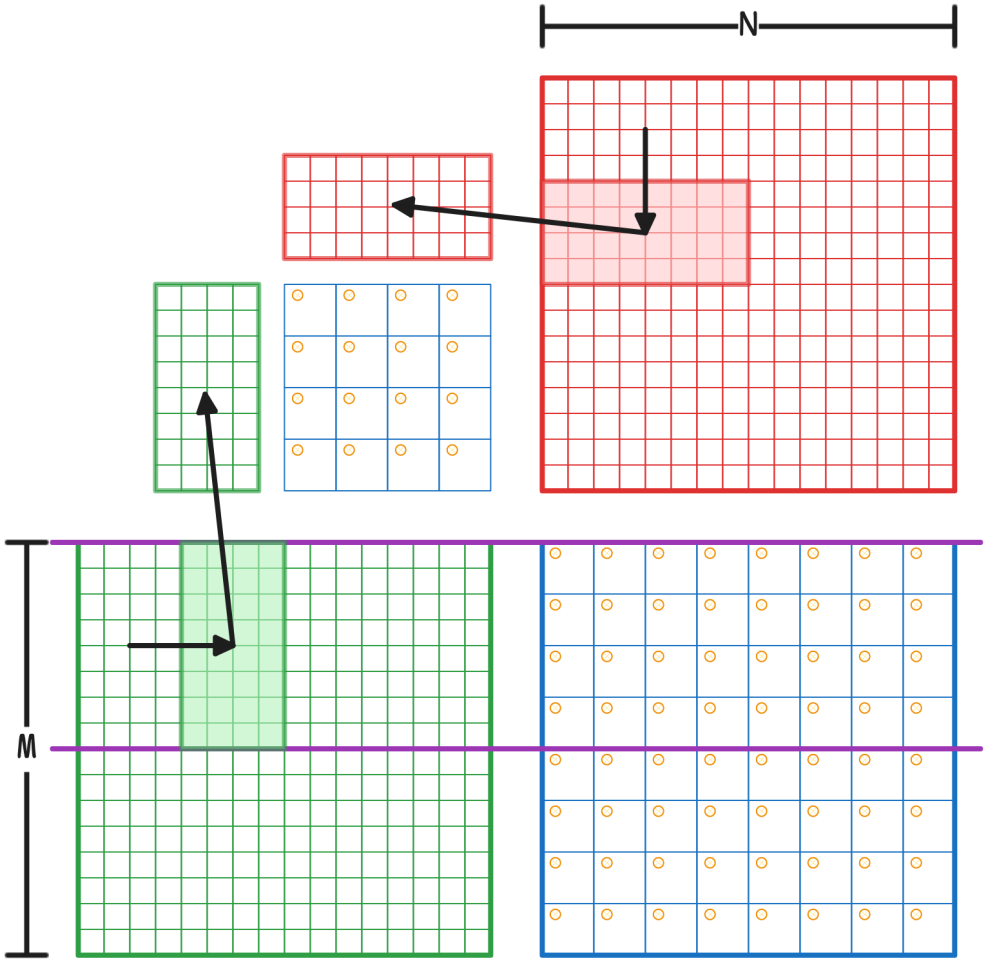
- Basic tiling assigns 1 element of the output to each thread.
- We can *coarsen* each thread, and assign a $TM \times TN$ portion of the output to each thread.
- We load larger chunks into SMEM, increasing data reuse and AI.
- One loaded A value can update several columns; one loaded B value can update several rows.



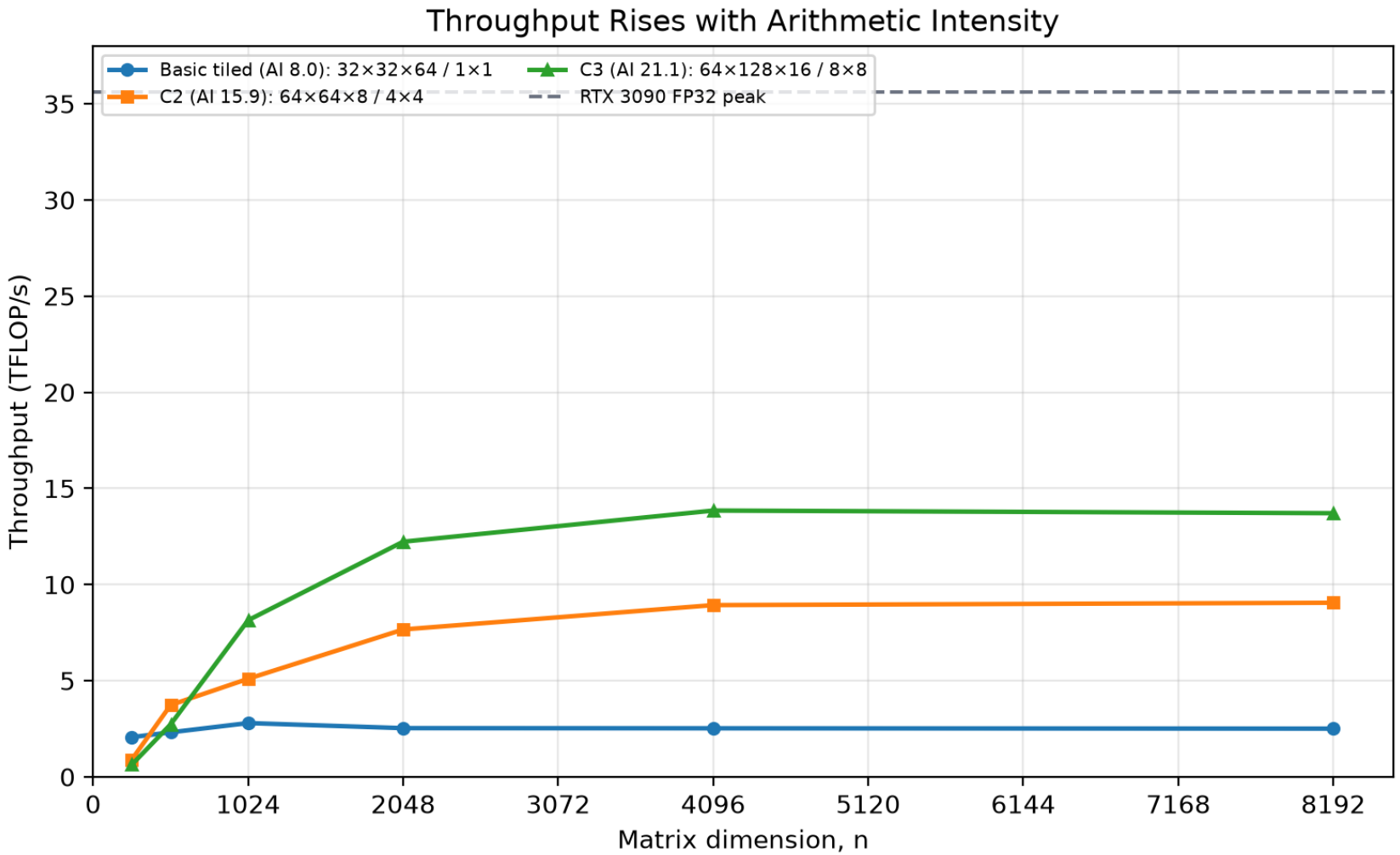
Coarsened SGEMM visualization



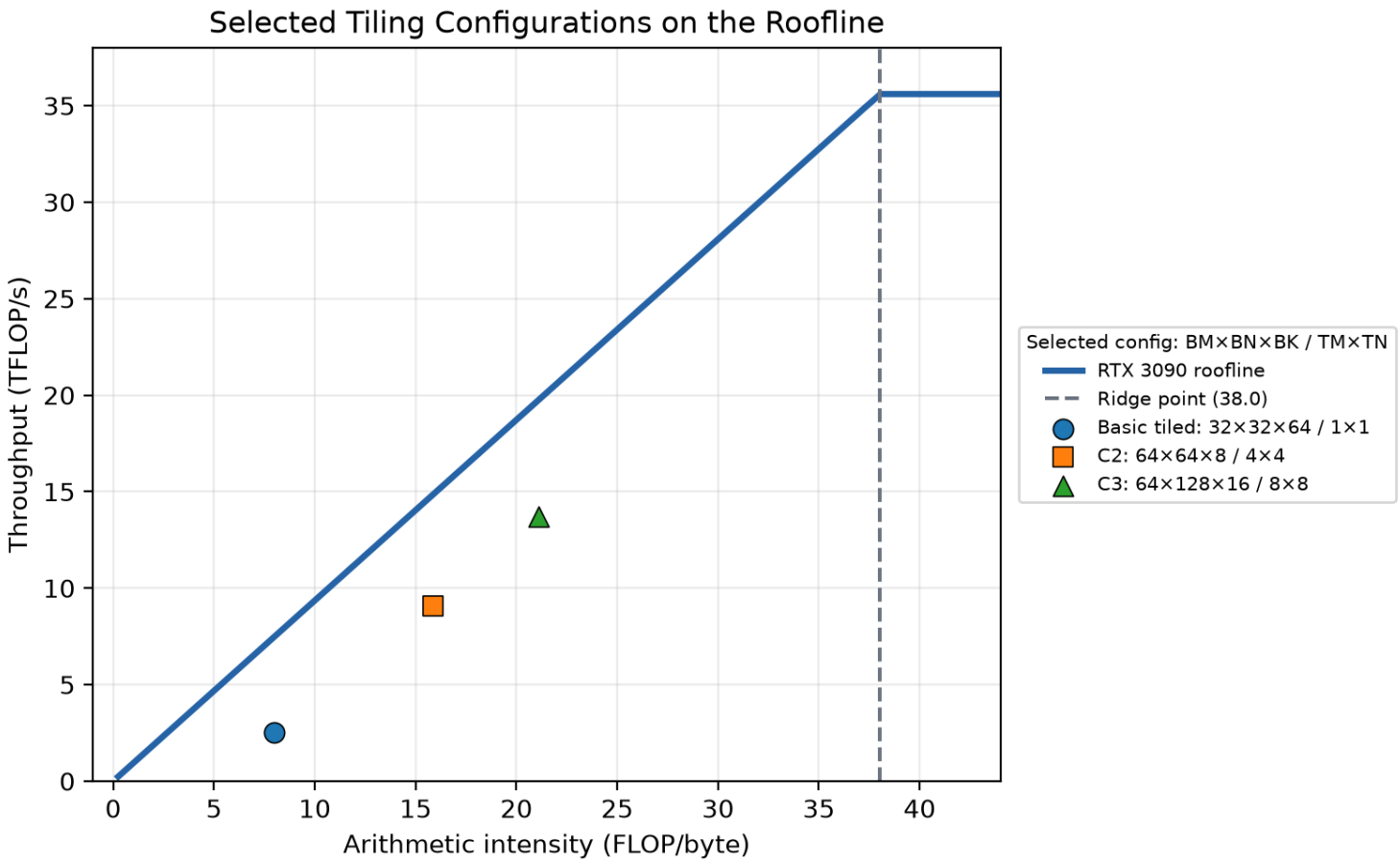
Coarsened SGEMM visualization



Higher-AI tiles tend to raise throughput

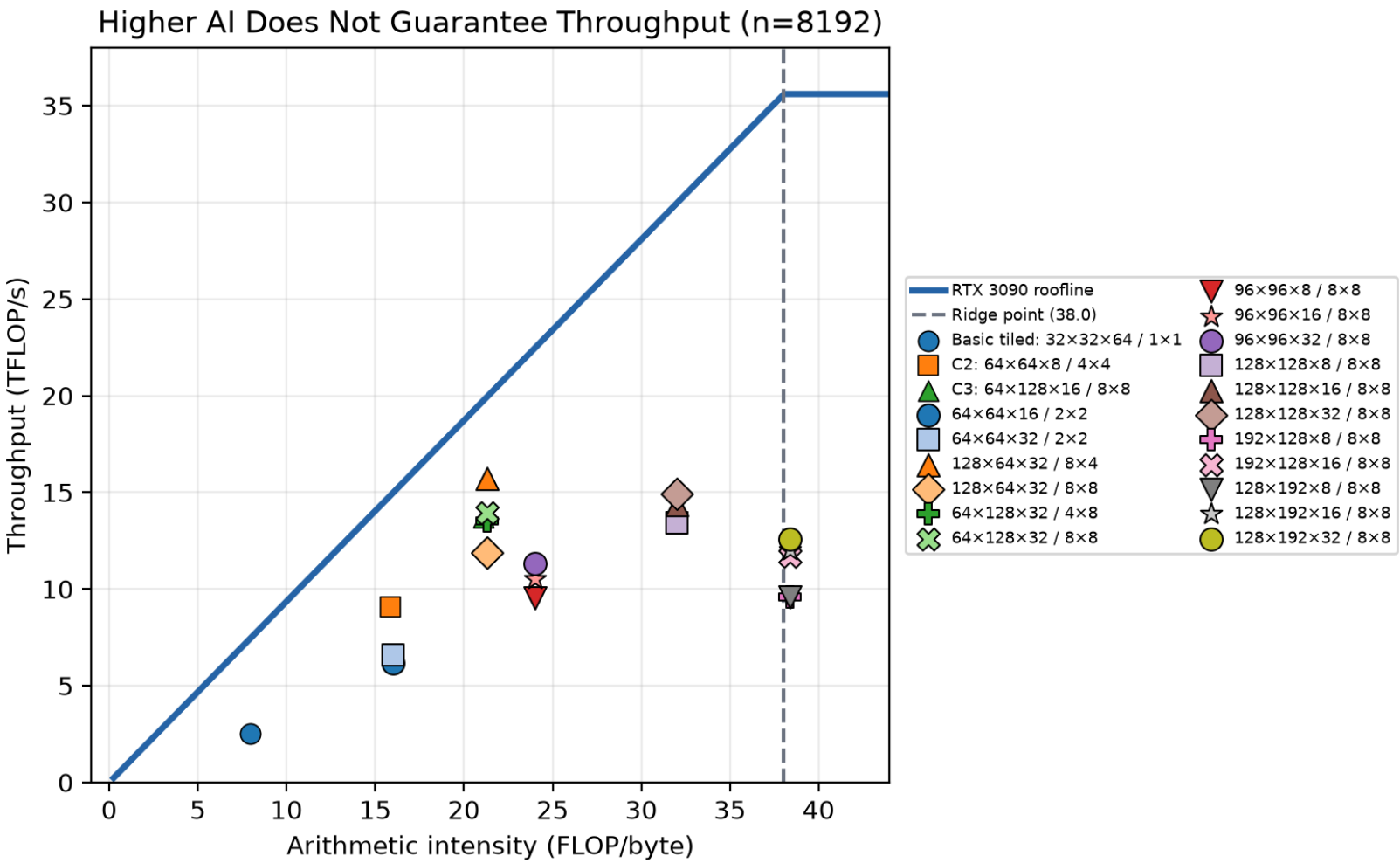


Tiling configurations on the roofline



Reminder that the Roofline Model only tells us the upper bound!

Higher AI does not guarantee higher throughput



The roofline is an upper bound, not a performance prediction.

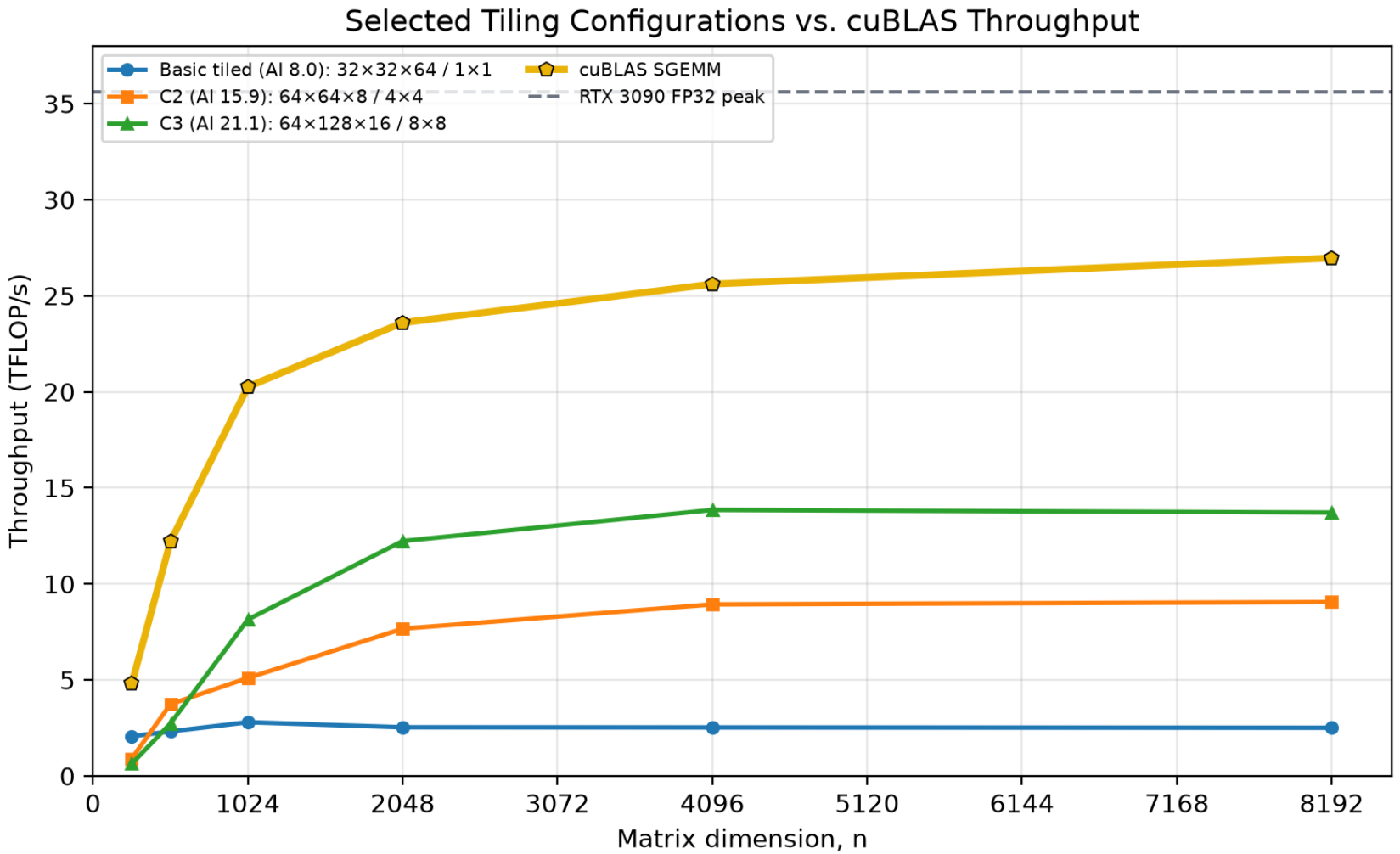
We have only scratched the surface: cuBLAS

- cuBLAS is NVIDIA's architecture-tuned linear algebra library and the practical performance target for our SGEMM kernels.
- At $n = 8192$, cuBLAS reaches 26.96 TFLOP/s, while the highest-AI selected configuration (C3) reaches 13.71 TFLOP/s, or 50.8% of cuBLAS's throughput.
- cuBLAS completes the multiplication in 40.8 ms versus 80.2 ms for C3, about 1.97× faster.

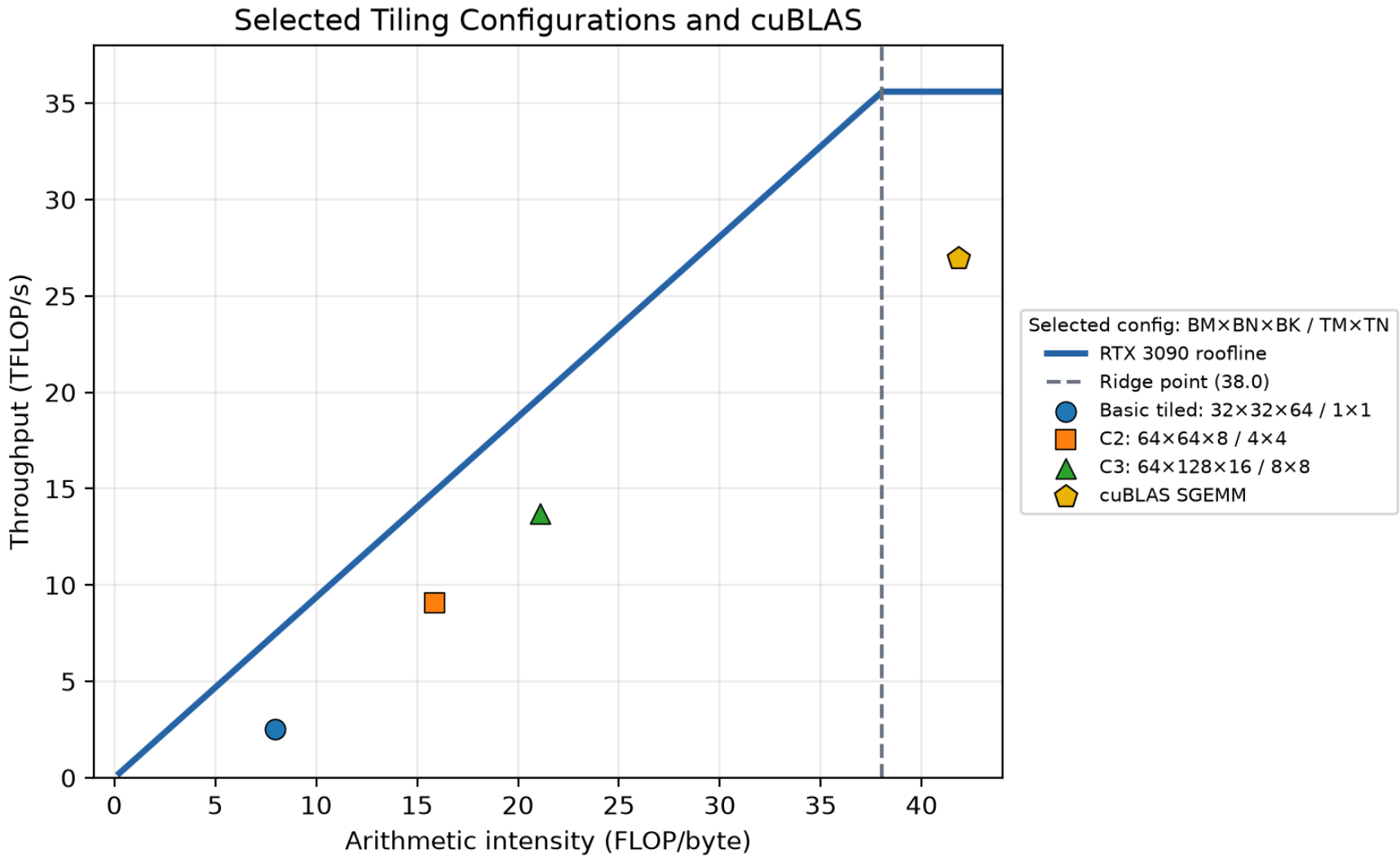
The remaining gap

Production kernels combine deeper register and warp tiling, vectorized memory access, pipelined loads, and architecture-specific instruction scheduling.

cuBLAS remains the performance target



cuBLAS on the roofline



cuBLAS arithmetic intensity is estimated from the profiled 256×128 output-tile shape.

We've come a long way

